

TINY MACHINE LEARNING · ETTML

ETTML – præsentationsnoter og fagligt opslagsværk

Præsentationsflow, kerneteori, projektresultater og
dokumenteret kodekort.

Tabletudgave · brug dokumentets globale begrebsindeks til hurtigt opslag

Del A følger PowerPoint-præsentationen 1:1. Hver slide har sin egen sektion og starter på en ny PDF-side. Brug først det korte taleforløb og de vigtige pointer. Brug uddybningen og kodehenvisningerne, hvis du har brug for flere detaljer. **Del B** er det dybe begrebsopslag.

I PDF'en er alle blå emnenavne klikbare. Tryk eksempelvis på **StandardScaler** under slide 12 for at hoppe direkte til teorien i del B. Brug derefter PDF-læserens tilbagepil for at vende tilbage til talekortet.

Del A – Præsentationsnoter slide for slide

Hurtig navigation

[Slide 1](#) · [Slide 2](#) · [Slide 3](#) · [Slide 4](#) · [Slide 5](#) · [Slide 6](#) · [Slide 7](#) · [Slide 8](#) · [Slide 9](#) · [Slide 10](#) · [Slide 11](#) · [Slide 12](#) · [Slide 13](#) · [Slide 14](#) · [Slide 15](#) · [Slide 16](#) · [Slide 17](#) · [Slide 18](#) · [Begrebsindeks](#)

Slide 1/18 – Tiny Machine Learning

Slidebudskab: Projektet er en lokal TinyML-løsning, der genkender fysiske gestusser på en Photon 2 med en ADXL343.

Kort taleforløb:

"Mit projekt hedder *Gesture Recognition on the Edge*. Jeg har bygget et system, hvor en ADXL343 måler bevægelser, og en lille machine-learning-model på en Particle Photon 2 omsætter dem til kommandoer. Inferensen foregår lokalt på enheden. Jeg starter med at vise produktet og følger derefter dataene kronologisk fra sensor til resultat."

Vigtige pointer:

- **TinyML** er machine learning på ressourcebegrænset embedded hardware.
- **Edge inference** betyder, at nye input behandles lokalt frem for i cloud.
- Modellen trænes på en computer, men det færdige forward-pass kører på Photon 2.
- Projektet kombinerer embedded programmering, signalbehandling og machine learning.

Uddybning i del B: [TinyML](#) · [Edge computing](#) · [Inference](#)

Overgang:

"Før demonstrationen viser jeg kort den rækkefølge, jeg gennemgår projektet i."

Slide 2/18 – Agenda

Slidebudskab: Præsentationen følger dataenes vej fra fysisk problem til målt resultat.

Kort taleforløb:

“Jeg går kronologisk gennem projektet: først problem og demo, derefter hardware og dataindsamling, så preprocessing og features, model og deployment og til sidst resultater og begrænsninger. Flowet er problem → data → features → model → resultat.”

Vigtige pointer:

1. Projekt og demonstration.
2. Hardware og data.
3. Features.
4. Model og deployment.
5. Resultater og næste iteration.

Denne rækkefølge gør det muligt at forklare hvert designvalg ud fra det foregående trin. De planlagte spørgepauser ligger efter hardware, data, model og resultater.

Overgang:

“Jeg begynder derfor med det konkrete problem, systemet skal løse.”

Slide 3/18 – Projektintroduktion

Slidebudskab: Systemet klassificerer fem kendte bevægelsesklasser og omsætter fire aktive gestusser til observerbare blackjack-kommandoer.

Kort taleforløb:

“Formålet var at lave en gestusbaseret brugergrænseflade, som fungerer lokalt på Photon 2. Sensorvinduet klassificeres som idle, ét tap, to taps, tre taps eller en venstre-højre-rystelse. De aktive klasser bliver til henholdsvis stand, hit, exit og split. Idle betyder, at systemet ikke skal udføre en handling.”

Vigtige pointer:

- `idle` → ingen kommando.
- `tap1` → `stand`.
- `tap2` → `hit`.
- `tap3` → `exit`.
- `shake_lr` → `split`.
- Outputtet er både en serial `EVENT`-linje og RGB-feedback.
- Dette er **supervised flerklassifikation**, fordi hvert træningseksempel har en kendt label.
- ML blev valgt, fordi klasserne beskrives af flere overlappende signalegenskaber; regler alene ville hurtigt blive skrøbelige.

Afgrænsning: Prototypen viser et fungerende end-to-end-system. Den dokumenterer ikke generalisering til nye brugere, fordi de rapporterede data kommer fra én operatør og én afsluttende session.

Uddybning i del B: [Problem og scope](#) · [Supervised learning](#) · [Klassifikation](#)

Overgang:

“Før jeg forklarer teknikken, viser jeg hele kæden i praksis.”

Slide 4/18 – Demo

Slidebudskab: En fysisk gestus bliver målt, klassificeret lokalt og vist som **EVENT** og RGB-feedback.

Kort taleforløb:

“Jeg udfører nu en gestus. ADXL343 sender målinger til Photon 2, firmwaren beregner features, og MLP'et giver fem klassescores. Når resultatet er sikkert og stabilt, udsender enheden en **EVENT** og viser RGB-feedback. Der anvendes ikke cloud-inference.”

Mens demoen kører:

1. Peg på den fysiske gestus.
2. Peg på det serielle **EVENT**-output.
3. Peg på RGB-feedbackken.
4. Nytt resultatet til den valgte blackjack-kommando.

Vigtige pointer:

- Demoen dokumenterer integrationen fra sensor til output.
- Den dokumenterer ikke alene modellens generaliserings-accuracy.
- Klassifikationen er lokal; computerens serielmonitor viser kun enhedens output.
- Hvis en gestus ikke accepteres, kan det skyldes confidence, stabilitetskravet, tap-kontrollen eller debounce — ikke nødvendigvis manglende sensorinput.

Kode: `Product/firmware/src/main.cpp`: `runInference()` og `updateDecision()`.

Overgang:

“Nu går jeg tilbage til begyndelsen af kæden og forklarer den hardware, der producerer inputdataene.”

Slide 5/18 – Hardware: ADXL343 og Photon 2

Slidebudskab: Photon 2 læser synkroniseret X-, Y- og Z-acceleration fra ADXL343 over I2C.

Kort taleforløb:

“Hardwaren består af en Particle Photon 2 og et digitalt tre-akset ADXL343-accelerometer. De kommunikerer over I2C. Sensoren er sat til 400 Hz og ± 16 g i full-resolution mode. Hver måling indeholder X, Y og Z, som omregnes fra rå signed 16-bit-tal til g.”

Vigtige pointer:

- Photon 2 anvender en 200 MHz Arm Cortex-M33 som hovedprocessor.
- **D0** er SDA, og **D1** er SCL.
- Firmwaren prøver I2C-adresserne **0x53** og **0x1D** og forventer device-ID **0xE5**.
- Sensorområdet er ± 16 g for at reducere risikoen for clipping ved hurtige taps.
- Den nominelle full-resolution scale factor er cirka **0,0039 g/LSB**.
- Sensorens Output Data Rate er 400 Hz; I2C-clock på 400 kHz er bussens bithastighed og ikke samplingfrekvensen.
- En stationær sensor viser ikke nødvendigvis nul: tyngdekraft, offset og støj er stadig til stede.

Kode:

- `readAdxlRawXYZ()` samler seks registerbytes til X/Y/Z.
- `readSampleG()` ganger råværdierne med **0,0039**.
- `initAdxlMeasurementMode()` konfigurerer sensoren.

Uddybning i del B: Accelerometer · Sensoropløsning og måleområde · I2C

Overgang:

“Da forbindelsen virkede, var næste opgave at forstå, hvordan de tre akser faktisk lå i min fysiske montering.”

Slide 6/18 – Fra antagelse til målt orientering

Slidebudskab: Datastrømmen var korrekt X/Y/Z, men den første fysiske fortolkning af akserne var forkert.

Kort taleforløb:

“Jeg antog først, at Y var op og ned. Dataanalysen viste, at den fysiske orientering i min montering var Y venstre-højre, X frem-tilbage og Z op-ned. Sensoren sendte stadig værdierne i X-, Y-, Z-rækkefølge; det var min fortolkning af retningerne, jeg rettede. Taps gav samtidig en markant stærkere respons på Z end på Y.”

Målt evidens:

- Ved idle var middelværdierne cirka $X = 0,04 \text{ g}$, $Y = 0,03 \text{ g}$, $Z = 0,91 \text{ g}$; tyngdekraften lå hovedsageligt på Z.
- For tap-klasserne var mean-centreret peak cirka $7,64 \text{ g}$ på Z mod $4,48 \text{ g}$ på Y.
- Tap-RMS var cirka $0,304 \text{ g}$ på Z mod $0,152 \text{ g}$ på Y.
- Z var peak-dominant i 10 af 15 tap-optagelser.
- Shake havde derimod nyttig vedvarende information på Y.

Designkonsekvens: Den endelige pipeline bruger X, Y og Z samt magnitude. Der foretages ikke en kunstig ombytning af Y- og Z-kolonnerne.

Vigtig præcision: Resultatet understøtter designændringen, men der blev ikke udført en kontrolleret akseablation. Derfor kan hele modelverbredringen ikke tilskrives Z alene.

Uddybning i del B: 3D-akser og fysisk orientering · Iterativ udvikling

Overgang:

“Med sensor og orientering på plads kan hele systemets dataflow nu vises fra venstre mod højre.”

Slide 7/18 – Samlet pipeline

Slidebudskab: Hver fysisk gestus gennemgår en fast kæde fra rå acceleration til en filtreret handling.

Kort taleforløb:

“ADXL343 måler X, Y og Z ved 400 Hz. Fire sekunder giver 1.600 samples per akse. Firmwaren fjerner middelværdien, beregner magnitude og 28 features, standardiserer dem og sender dem gennem MLP'et. Modellens output filtreres derefter med confidence, stabilitet, tap-kontrol og debounce, før en kommando udsendes.”

Pipeline:

gestus → X/Y/Z → 1.600-sample-vindue → mean removal + magnitude → 28 features → StandardScaler
→ MLP → fem scores → beslutningsfilter → EVENT + RGB

Vigtige pointer:

- Preprocessing og featureorden skal være identiske under træning og LIVE-inference.
- MLP'et klassificerer; beslutningslogikken afgør, om klassifikationen må blive til en handling.
- Træning foregår offline. Den deployede kæde anvender faste scaler- og modelparametre.
- Systemet er derfor en hybrid: lært klassifikation efterfulgt af deterministiske sikkerhedsregler.

Uddybning i del B: [Hele systemets dataflow](#) · [TinyML-toolchain](#)

Overgang:

“Det første praktiske trin i pipelinen var at få konsistente og labellede sensoroptagelser.”

Slide 8/18 – Dataindsamling

Slidebudskab: Rå sensorobservationer og en stationær baseline blev brugt til at skelne almindelig støj fra reel interaktion og til at etablere den endelige 400 Hz-kontrakt.

Kort taleforløb:

"Jeg begyndte med at observere de rå X-, Y- og Z-data ved stilstand. De er ikke nul på grund af tyngdekraft, offset og støj. En stationær baseline gav derfor et referencepunkt for normal variation. De tidlige guidede forsøg brugte 50 Hz, men hurtige taps blev bedre repræsenteret ved 400 Hz. Den endelige dataindsamling gemmer fire sekunder, altså 1.600 synkroniserede X/Y/Z-samples."

Baselineprincippet:

`motion threshold = max(0,05 g, 3 × standardafvigelsen for stationær magnitude)`

- Baseline-triggeren bestemmer kun, om bevægelsen afviger tydeligt fra stilstand.
- Den klassificerer ikke gestussen.
- Den ældre guidede baseline-rutine poller ved 50 Hz.
- Det endelige `pilot_v3`-datasæt og LIVE-modellen bruger 400 Hz.

Endelig capture:

- `TAP_SCOPE` sampler X/Y/Z i RAM ved 2,5 ms interval.
- Data udskrives først bagefter, så seriel tekst ikke forstyrrer samplingtimingen.
- Fire sekunder × 400 Hz = 1.600 samples.
- Der gemmes både signal, label, tempo, kraft, session og kvalitetsmetadata.

Kode: `startTapScope()` og `runTapScope()` i `main.cpp`; captureflowet i `randomized_capture_gui.py`.

Uddybning i del B: Fra rådata til baseline · Sampling · Hvorfor 400 Hz?

Overgang:

"Når et signal var optaget, skulle det kvalitetssikres, før det måtte blive en del af datasættet."

Slide 9/18 – Datasæt og kvalitetskontrol

Slidebudskab: Det rapporterede datasæt er lille og balanceret, og automatiske kvalitetsgates beskytter labels uden at fjerne risikoen for bias.

Kort taleforløb:

”Rapportens model bruger 25 accepterede optagelser: fem fra hver af de fem klasser. Trials blev randomiseret på tværs af klasser og udførelsesbetingelser. Captureprogrammet kontrollerede blandt andet clipping, bevægelse under idle, impact count for taps og varighed og RMS for shake. Det forbedrer labelkvaliteten, men meget stram filtrering kan også skabe unaturligt rene data.”

Klassernes fordeling:

- `idle` : 5
- `tap1` : 5
- `tap2` : 5
- `tap3` : 5
- `shake_1r` : 5

Automatiske kvalitetsgates:

- Clipping: afvis ved rå akse på mindst `15,5 g`.
- Idle: dynamisk peak-magnitude højst `0,15 g`.
- Tap: aksepeak mindst `0,40 g` og præcis forventet antal impacts.
- Impact: dynamisk magnitude mindst `0,35 g`; mere end `150 ms` mellem aktive områder giver et nyt event.
- Shake: dynamisk RMS mindst `0,10 g` og aktiv varighed mindst `300 ms`.

Begrænsninger: Én operatør, én afsluttende session og kun fem eksempler per klasse. Klassebalance fjerner ikke bruger-, session- eller monteringsbias.

Kode: `metrics()`, `validate()` og `save()` i `Product/ml/randomized_capture_gui.py`.

Uddybning i del B: [Datasættet](#) · [Kvalitetskontrol](#) · [Dataset bias](#)

Overgang:

”De rå tidsserier er for store og tidsafhængige som direkte input til mit lille MLP, så næste trin er feature engineering.”

Slide 10/18 – Fra tidsserie til 28 features

Slidebudskab: Hver 4-sekunders optagelse komprimeres fra 4.800 rå akseverdier til 28 forklarlige modelinputs.

Kort taleforløb:

“Først beregner jeg magnitude fra X, Y og Z. Derefter fjerner jeg middelværdien separat fra X, Y, Z og magnitude for at reducere statisk offset. Fra hver af de fire kanaler udtrækker jeg syv statistiske egenskaber. Fire gange syv giver 28 features.”

De fire kanaler: X, Y, Z og $\text{magnitude} = \sqrt{X^2 + Y^2 + Z^2}$.

Syv features per kanal:

1. Standardafvigelse — signalets variation.
2. Minimum — største negative udsving efter centrering.
3. Maksimum — største positive udsving.
4. Range — maksimum minus minimum.
5. Energi — $\text{mean}(x^2)$, ikke fysisk energi i joule.
6. Peak count — antal tydelige lokale peaks.
7. Maximum absolute difference — hurtigste ændring mellem nabosamples.

$4 \text{ kanaler} \times 7 \text{ features} = 28 \text{ features}$

Hvorfor: Det reducerer inputdimension, modelstørrelse, RAM og beregning. Tradeoffet er, at noget af den præcise tidslige rækkefølge går tabt.

Vigtig præcision: En feature er en numerisk egenskab udledt af inputtet. Den bestemmer ikke alene outputtet; modellen lærer, hvordan alle 28 features skal vægtes og kombineres.

Uddybning i del B: [Preprocessing og features](#) · [Magnitude](#) · [Feature engineering](#)

Overgang:

“På næste slide viser jeg den konkrete Python-kode, der udfører denne komprimering.”

Slide 11/18 – Kodefokus: Feature extraction

Slidebudskab: Koden gennemløber fire kanaler, mean-centrerer hver kanal og tilføjer syv features i en fast rækkefølge.

Kort taleforløb:

"`extract_features_for_window()` starter med X, Y og Z og tilføjer magnitude som fjerde kanal. For hver kanal konverteres data til et numerisk array, middelværdien trækkes fra, og `channel_features()` returnerer de syv statistikker. Resultaterne føjes til én liste, som derfor ender med 28 tal."

```
channels = ["ax", "ay", "az"]
if use_mag:
    channels = channels + ["mag"]

feats = []
for c in channels:
    x = window_df[c].to_numpy(dtype=np.float32)
    x = x - np.mean(x)
    feats.extend(channel_features(x))
```

Linje for linje:

- `channels` fastlægger kanalrækkefølgen.
- `to_numpy(dtype=np.float32)` giver et numerisk array med embedded-relevant præcision.
- `x - np.mean(x)` fjerner kanalens gennemsnit i dette vindue.
- `channel_features(x)` beregner de syv tal.
- `extend` lægger alle syv tal efter hinanden; det er ikke det samme som at tilføje en indlejret liste.

Kodekontrakt: Python og C++ skal bruge rækkefølgen X-features → Y-features → Z-features → magnitude-features. Ændret orden giver de lærte vægte forkerte input.

Kodehenvisninger: `extract_features_for_window()` og `channel_features()` i `train.py`; `extractFeatures()` og `channelFeatures()` i `main.cpp`.

Overgang:

"De 28 tal har meget forskellige størrelser, så før de går ind i netværket, skal de skaleres."

Slide 12/18 – Skalering og neural model

Slidebudskab: Hver feature standardiseres, før et lille MLP kombinerer de 28 inputs til fem classesandsynligheder.

Kort taleforløb:

“StandardScaler lærer et gennemsnit og en skala for hver af de 28 featuredimensioner fra træningsdata. En ny feature omregnes som $(x - \mu) / \sigma$. De standardiserede inputs sendes gennem et MLP med 28 inputs, skjulte lag på 32 og 16 neuroner og fem outputs. De skjulte lag bruger ReLU, og outputlaget bruger softmax.”

StandardScaler:

$$z = (x - \mu) / \sigma$$

- μ og σ læres kun fra træningsdata.
- Hver featuredimension har sine egne værdier.
- De 28 middelværdier og 28 skalaer eksporteres til C++.

Et neuron:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

- Inputs multipliceres med lærte vægte.
- Bias forskyder resultatet.
- ReLU giver $\max(0, z)$ i de skjulte lag.
- Softmax omsætter de fem logits til positive scores, der summerer til 1.

Arkitektur og størrelse: $28 \rightarrow 32 \rightarrow 16 \rightarrow 5$, i alt 1.541 lærte vægte og biases.

Hvorfor denne model: Den er lille, hurtig og kan udtrykke ikke-lineære kombinationer af features. Med kun 25 eksempler er der dog stadig betydelig overfitting-risiko.

Uddybning i del B: StandardScaler · Neuron, vægte og bias · ReLU · Softmax

Overgang:

“Netværkets vægte skal først læres og derefter vurderes på data, som ikke deltog i træningen.”

Slide 13/18 – Træning og evaluering

Slidebudskab: Modellen trænes supervised med backpropagation og evalueres både med et fast stratificeret holdout og fem skiftende testfolds.

Kort taleforløb:

"Alle optagelser har labels, så modellen trænes supervised. I et stratificeret 80/20-holdout trænes på 20 optagelser og testes på fem — én fra hver klasse. Seed 42 gør opdelingen reproducerbar. Som supplerende stabilitetskontrol anvendte jeg stratificeret 5-fold cross-validation, hvor hver gruppe er testdata én gang."

Under træning:

1. Forward-pass beregner scores.
2. Loss måler forskellen mellem prediction og korrekt label.
3. Backpropagation beregner gradienter baglæns gennem lagene.
4. Optimizeren opdaterer vægte og biases.
5. Processen gentages op til 500 iterationer.

80/20-holdout: 20 train + 5 test. Én fejl ændrer accuracy med 20 procentpoint.

5-fold-CV: Fem træninger med fold-accuracy 60 %, 60 %, 100 %, 80 %, 80 %; gennemsnit 76 %.

Seed 42: En vilkårlig startværdi til den pseudo-tilfældige blanding. Den gør eksperimentet reproducerbart, men forbedrer ikke modellen.

Vigtig præcision: Den nuværende `train.py` udfører det faste holdout. CV-resultatet kommer fra en separat kontrolberegning med ny scaler og ny MLP i hver fold.

Uddybning i del B: Backpropagation · Stratificeret holdout · Cross-validation · Seed 42

Overgang:

"På næste slide viser jeg de kodelinjer, som udfører split, skalering og træning uden data leakage."

Slide 14/18 – Kodefokus: Split, scaler og træning

Slidebudskab: Testdata holdes ude af både scaler-fit og modeltræning.

Kort taleforløb:

”`train_test_split` laver det stratificerede 80/20-split med seed 42. `fit_transform` på træningsdata lærer scalerens værdier og anvender dem. Testdata får kun `transform`, så de ikke påvirker preprocessing. Derefter træner `model.fit()` MLP'et på de 20 skalerede træningseksempler, og `predict()` udfører inference på de fem testeksempler.”

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y_enc,
    test_size=0.2,
    random_state=42,
    stratify=y_enc,
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

model = train_model(cfg, X_train_s, y_train)
y_pred = model.predict(X_test_s)
```

Vigtige pointer:

- `stratify=y_enc` bevarer klassefordelingen.
- `random_state=42` låser den reproducerbare opdeling.
- `fit_transform(train)` lærer kun fra train.
- `transform(test)` genbruger train-statistikken.
- Hvis scaler blev fitted på alle 25 eksempler, ville det være data leakage.
- `model.fit()` skjuler forward-pass, loss, backpropagation og vægtopdateringer bag ét API-kald.
- Konfigurationens batch size er 32, men der er kun 20 træningseksempler; scikit-learn reducerer derfor den effektive batch til 20.

Kode: `main()` og `train_model()` i `Product/ml/train.py`.

Overgang:

”Når modellen er trænet, skal scaler og lærte parametre flyttes fra Python til Photon 2.”

Slide 15/18 – Deployment på Photon 2

Slidebudskab: Træningen foregår offline, mens Photon 2 udfører et lille C++-forward-pass med faste parametre og efterfølgende beslutningsfiltre.

Kort taleforløb:

"`export_model.py` skriver scalerens værdier, klasser, vægte og biases til `model_data.h` og `model_data.cpp`. På Photon 2 standardiseres de 28 features, hvorefter netværket beregner lagene $28 \rightarrow 32 \rightarrow 16 \rightarrow 5$. Det er inference med faste parametre; der udføres ikke backpropagation eller selvtræning på enheden."

Deploymentkæde:

Python-data → features → scaler + MLP-træning → C++-arrays → Particle-build → flash → LIVE-inference

Hvorfor offline træning:

- Backpropagation kræver ekstra aktiveringer, gradienter, RAM, beregning og energi.
- LIVE-data har ikke automatisk korrekte labels.
- Offline træning gør datasplit, metrics og modelversion lettere at kontrollere.
- On-device training er ikke principielt umulig, men kræver en anden arkitektur og er ikke nødvendig i dette projekt.

Beslutningsfiltre: score mindst `0,75`, tre ens predictions, fysisk tap-kontrol og `4.000 ms` debounce.

Ressourcer: Seneste build bruger `27.950 B` flash og `46.686 B` RAM. Selve model-forward-pass blev målt til cirka `0,35 ms`.

Kode: `Product/ml/export_model.py`, `Product/firmware/src/model_data.cpp` og `main.cpp`.

Uddybning i del B: Python-model til C++ · Offline og on-device training · Designbegrundelse

Overgang:

"Næste kodeudsnit viser, hvordan enheden går fra 28 features til en godkendt `EVENT`."

Slide 16/18 – Kodefokus: Inference og beslutning

Slidebudskab: Modelkaldet giver fem scores; firmwarelogikken gør først den bedste sikre og stabile score til en handling.

Kort taleforløb:

"`runInference()` beregner først 28 features. `model_infer()` standardiserer dem og udfører forward-pass til fem softmax-scores. En løkke finder den højeste score. `updateDecision()` kræver derefter tilstrækkelig confidence, tre ens predictions, korrekt tap count og udløbet debounce, før der udsendes et event."

```
extractFeatures(features);
tinyml_model::model_infer(features, 28, scores, 5);

int bestIdx = 0;
for (size_t i = 1; i < 5; ++i) {
    if (scores[i] > scores[bestIdx]) bestIdx = (int)i;
}

const int eventClass = updateDecision(
    bestIdx, scores[bestIdx], millis());
```

Beslutningsrækkefølge:

28 features → scaler + MLP → fem scores → argmax → score $\geq 0,75$ → tre ens → tap-kontrol → 4 s debounce → EVENT + RGB

Vigtige pointer:

- `argmax` vælger indeks for den højeste score. `return -1` betyder "ingen godkendt handling endnu"; det er ikke en modelklasse.
- Filtrene reducerer falske events, men øger responsforsinkelsen og kan afvise korrekte ustabile predictions.
- Python-accuracy måler MLP'et. Den komplette LIVE-adfærd inkluderer yderligere regler og har derfor ikke automatisk samme event-accuracy.
- Inferenstimingtallet måler kun `model_infer()`, ikke feature extraction. Der findes endnu ikke en automatisk golden-vector-test mellem Python og C++.

Kode: `runInference()`, `countImpactEvents()` og `updateDecision()` i `main.cpp`; `model_infer()` i `model_data.cpp`.

Uddybning i del B: Forward-pass · Firmware og beslutningslogik · Confidence threshold · Debounce

Slide 17/18 – Resultater

Slidebudskab: Prototypen virker end-to-end og opnåede lovende pilotresultater, men testgrundlaget er meget lille.

Kort taleforløb:

”På det stratificerede holdout blev fire af fem testoptagelser klassificeret korrekt, altså 80 %. Tap1 blev forvekslet med tap2. Macro precision var 70 %, recall 80 % og F1 73,3 %. Femfolds-cross-validation gav 76 % i gennemsnit. Model-forward-pass tog cirka 345 mikrosekunder i gennemsnit, og den registrerede LIVE-test havde nul sensor-read-fejl.”

Dokumenterede tal:

- Holdout accuracy: 80 % = 4/5.
- Macro precision: 70,0 %.
- Macro recall: 80,0 %.
- Macro F1: 73,3 %.
- 5-fold-CV: 76 % = 19/25 fold-predictions.
- Fejl: tap1 → tap2 i holdout-splittet.
- Model-inference: cirka 345 μ s gennemsnit og 364 μ s maksimum i den registrerede LIVE-test.
- Firmware-build: 27.950 B flash og 46.686 B RAM.
- Registrerede sensor-read-fejl i LIVE-testen: 0.

Fortolkning: Én fejl blandt fem testeksempler ændrer accuracy med 20 procentpoint. Resultaterne viser feasibility og en fungerende prototype, ikke et præcist estimat for nye brugere.

Vigtig skelnen: Confusion matrix og klassifikationsmetrics gælder MLP'ets predictions på holdout-data. Den fysiske demo viser den samlede hybridpipeline med beslutningsfiltre.

Uddybning i del B: Projektets resultater · Confusion matrix · Accuracy · Precision · Recall · F1

Overgang:

”Til sidst adskiller jeg rapportens evaluerede snapshot fra den data, der er indsamlet bagefter.”

Slide 18/18 – Aktuel datastatus

Slidebudskab: Der findes nu mere data, men de rapporterede metrics må fortsat knyttes til det balancerede 25-optagelses-snapshot.

Kort taleforløb:

”Rapportens dokumenterede model og metrics er baseret på 25 balancerede optagelser. Repository indeholder nu 49 kontraktkompatible kandidater: fem idle, fem tap1, 14 tap2, 20 tap3 og fem shake. De ekstra data er ubalancerede og er endnu ikke brugt til de rapporterede resultater. Næste kontrollerede iteration er derfor at indsamle de manglende klasser, kuratere og balancere, retrainere og gennemføre en ny blind evaluering.”

Status:

- **Rapportsnapshot:** 25 optagelser, fem per klasse, balanceret, trænet, evalueret og deployet.
- **Aktuel kandidatpulje:** 49 kontraktkompatible optagelser.
- Fordeling: `idle 5 · tap1 5 · tap2 14 · tap3 20 · shake_lr 5`.
- De 24 ekstra optagelser ligger kun i tap2 og tap3.
- Flere optagelser er ikke automatisk bedre evidens, hvis klasse-, bruger- og sessionsvariation mangler.

Næste iteration:

1. Indsaml flere idle-, tap1- og shake-optagelser samt nye brugere/sessioner.
2. Kurater og balancer datasættet.
3. Fastlæg et nyt uafhængigt split.
4. Retrain og eksportér modellen.
5. Gentag offline metrics og end-to-end LIVE-test.

Afsluttende konklusion:

”Projektet demonstrerer en fungerende lokal TinyML-pipeline fra fysisk gestus til observerbar handling. De vigtigste tekniske bidrag er den målte forståelse af sensororienteringen, den konsekvente 28-feature-kontrakt og deployment af et rigtigt MLP-forward-pass på Photon 2. Den største begrænsning er datamængde og repræsentativitet, så næste skridt er bedre data og en ny blind evaluering — ikke at overfortolke de nuværende 80 %.”

Uddybning i del B: Aktuel datapulje · Begrænsninger og næste eksperiment · Stærk konklusion

Del B – Fagligt opslagsværk

Sådan bruges opslagsværket

Hvert fagligt begreb har sin egen overskrift. Under overskriften står først en enkel forklaring i hele sætninger og derefter de vigtigste pointer som bullets. Hvor det er relevant, står der også et konkret eksempel fra projektet og ekstra teori.

Et kort svar kan normalt bygges sådan:

“Begrebet betyder ... Det bruges, fordi ... I mit projekt bruges det til ... En vigtig begrænsning er ...”

Globalt begrebsindeks

Akut hjælp

- Tilbage til slide-for-slide-talekort
- Hvis jeg går blank
- Projektet på 60 sekunder
- Hele systemets dataflow

Data og signaler

- Accelerometer
- Sensoropløsning, måleområde og enheder
- Sampling og samplingfrekvens
- Nyquist-frekvens og aliasing
- Filtrering
- Vinduer og sliding windows
- Datasæt, labels og klassebalance
- Metadata
- Dataset bias og repræsentativitet
- Preprocessing
- Magnitude
- Feature engineering
- Tidsdomæne og frekvensdomæne
- Feature selection

Machine learning

- Machine learning, deep learning og AI
- Supervised learning
- Unsupervised learning
- Klassifikation, regression og anomaly detection
- StandardScaler
- MLP
- Neuron, vægte og bias
- ReLU
- Softmax
- Forward-pass
- Loss-funktion
- Backpropagation
- Gradient descent og optimizer
- Epoch, batch og iteration
- Konvergens
- Decision boundary
- Random forest og beslutningstræer
- Convolution
- 1D-CNN
- Train, validation og test
- Overfitting og generalisering
- Data leakage

Evaluering

- Confusion matrix
- TP, FP, FN og TN
- Accuracy
- Precision
- Recall
- F1-score
- Macro average
- Cross-validation
- Projektets resultater

Embedded TinyML

- TinyML
- Edge computing og lokal inference

- Microcontroller kontra computer
- Python-model til C++
- TinyML-toolchain
- emlearn
- AlFES
- Edge Impulse
- CMSIS
- Offline og on-device training
- Confidence threshold
- Beslutningsstabilisering
- Debounce
- Non-blocking firmware
- Flash og RAM
- Latency og responstid
- Kvantisering
- Pruning
- Low-power-design
- K-means anomaly detection
- GMM
- Autoencoder

Projektforsvar

- Problem, scope og kursuskrav
- Begrænsninger og næste eksperiment
- Iterativ ML-udvikling
- AI-brug
- Kildegrundlag
- Kursusmateriale og projektkrav
- ADXL343-datablad
- Photon 2-dokumentation
- StandardScaler-dokumentation
- MLPClassifier-dokumentation
- Hands-On Machine Learning
- Machine Learning with PyTorch and Scikit-Learn
- Valg af kildetype
- Kodekort og dokumenterede henvisninger
- Spørgsmålszoner

1. Hvis jeg går blank

Min svarmodel

1. **Definition:** Hvad betyder begrebet?
2. **Formål:** Hvorfor bruger man det?
3. **Projekt:** Hvordan bruges det i min løsning?
4. **Begrænsning:** Hvad skal man være kritisk overfor?

Sætninger jeg må bruge

- "Giv mig lige et øjeblik til at strukturere mit svar."
- "Sådan som jeg forstår spørgsmålet, spørger I om ..."
- "Jeg starter med det grundlæggende princip."
- "I mit projekt kommer det konkret til udtryk ved ..."
- "Jeg kan ikke huske den præcise detalje, men princippet er ..."
- "Det målte jeg ikke direkte. Jeg ville undersøge det ved at ..."
- "Jeg vil nødig gætte. Det, jeg kan forklare sikkert, er ..."
- "Jeg vender lige tilbage til dataflowet, hvor jeg var nået til ..."

Mine fem holdepunkter

KRAV → DATA → FEATURES → MODEL → RESULTAT

2. Projektet på 60 sekunder

Jeg har bygget en gestusbaseret brugergrænseflade på en Particle Photon 2. En ADXL343-accelerationssensor måler bevægelse på X-, Y- og Z-aksen ved 400 Hz. Systemet opsamler fire sekunders data, fjerner signalernes gennemsnit og beregner 28 statistiske features. De bliver standardiseret og sendt gennem et lille neuralt netværk, som klassificerer vinduet som idle, ét tap, to taps, tre taps eller en venstre-højre-rystelse. Firmwaren filtrerer usikre og ustabile predictions, hvorefter en godkendt gestus bliver omsat til en blackjack-kommando og RGB-feedback. Hele inferencen foregår lokalt på microcontrolleren. Prototypen fungerer end-to-end, men datasættet er for lille til at dokumentere generalisering til nye brugere.

Ultrakort version

"Jeg har bygget en lokal TinyML-gestusklassifikator på en Photon 2. Den omdanner accelerometerdata til 28 features, klassificerer dem med et lille MLP og viser den godkendte kommando via USB og RGB-LED."

3. Hele systemets dataflow

fysisk gestus

- ADXL343 måler `ax`, `ay`, `az` ved 400 Hz
- 4 sekunders vindue med 1.600 samples
- gennemsnittet fjernes fra X, Y og Z
- magnitude beregnes
- 7 features beregnes for hver af 4 kanaler = 28 features
- StandardScaler bruger træningssættets middelværdi og spredning
- MLP med lagene 28–32–16–5
- softmax giver fem klassescores/sandsynligheder
- confidence, tre ens predictions, tap-kontrol og debounce
- `EVENT` over USB og RGB-feedback

Hovedpointen

Samme preprocessing, featureorden, scaler og modelparametre skal anvendes under træning i Python og inference i C++. Ellers modtager den deployede model andre input end dem, den blev trænet på.

4. Problem, scope og kursuskrav

Problem

Kan en lille model på en Particle Photon 2 skelne mellem fem accelerometerbaserede tilstande og omsætte dem til kommandoer lokalt?

Hvorfor klassifikation?

Outputtet er én af fem på forhånd definerede kategorier. Et kontinuerligt tal ville være regression, og registrering af noget ukendt eller unormalt ville være anomaly detection.

Hvorfor ML frem for kun regler?

Gestusser varierer i hastighed, kraft, timing og retning. Et regelsystem kan godt tælle tydelige taps, men bliver hurtigt skrøbeligt, når hele bevægelsesmønstret varierer. Modellen lærer kombinationer af features, mens regler efter modellen stabiliserer outputtet.

Machine learning, deep learning og kunstig intelligens

Kunstig intelligens er det brede område, hvor computere udfører opgaver, der forbindes med intelligent adfærd. Machine learning er en del af AI, hvor et system lærer mønstre fra data frem for kun at følge håndskrevne regler. Deep learning er en del af machine learning og bruger neurale netværk med flere lag.

- AI er den bredeste samlebetegnelse.
- Machine learning lærer parametre fra data.
- Deep learning anvender neurale netværk med flere lag.
- Et lille MLP kan kaldes et neuralt netværk, men "deep" er ikke en præcis garanti for, at modellen er avanceret.
- Valget bør bestemmes af opgaven, datamængden og hardwarebegrænsningerne.

I mit projekt: Der bruges supervised machine learning med et lille MLP. Systemet indeholder også traditionelle regler efter modellen.

Edge computing og lokal inference

Edge computing betyder, at behandling udføres tæt på datakilden i stedet for at sende alle rå data til en central cloudtjeneste. TinyML er en særlig ressourcebegrænset form for edge AI, ofte på microcontrollere.

- Lokal behandling kan fungere uden netværk.
- Der sendes færre rå data, hvilket kan forbedre privatliv og reducere kommunikationsenergi.
- Kommunikationslatency kan undgås.
- Enheden har til gengæld begrænset RAM, flash, regnekraft og energi.
- Lokal inference betyder ikke automatisk, at hele systemet har lav responstid; dataopsamlingen kan stadig dominere.

I mit projekt: Accelerometerdata forlader ikke enheden for at blive klassificeret. Photon 2 udfører selv featureberegning og forward-pass.

Scope

Projektet implementerer gestusinterfacet og kommandoerne til en blackjack-simulation. Det er ikke et komplet blackjack-spil.

Kursuskrav opfyldt i projektet

- Particle Photon 2 som embedded platform.
- Lokal ADXL343-sensor.
- Egne indsamlede og labellede data.
- Preprocessing af sensormålinger.
- ML-klassifikation.

- Lokal inference på enheden.
- Observerbart output via USB og RGB-LED.
- Kode, data og dokumentation i repository.

5. Hardware og sampling

Komponenter

- **Particle Photon 2:** Microcontroller-plattformen, som samler, beregner features og kører modellen.
- **ADXL343:** Tre-akset accelerometer forbundet over I2C.
- **RGB-LED:** Lokalt og synligt output.

Accelerometer

Det måler acceleration langs tre akser. Målingerne indeholder både bevægelse og tyngdeaccelerationens projektion på akserne.

En stationær sensor viser derfor ikke nødvendigvis $x = 0$, $y = 0$, $z = 0$. Afhængigt af orienteringen vil tyngdekraften være fordelt på en eller flere akser, og den samlede magnitude vil typisk ligge omkring 1 g . Små udsving under stilstand kan skyldes sensorstøj, offset, vibrationer og afrunding.

ADXL343 i projektet

- Digitalt, tre-akset accelerometer.
- Forbundet til Photon 2 over I2C med $D0 = SDA$ og $D1 = SCL$.
- Firmwaren undersøger adresserne $0x53$ og $0x1D$ og forventer device-ID $0xE5$.
- Sensorens Output Data Rate er sat til 400 Hz.
- Måleområdet er sat til $\pm 16\text{ g}$ i full-resolution mode.
- Den nominelle omregning er cirka $0,0039\text{ g}$ per rå LSB.
- De seks databytes fra sensoren samles til signed 16-bit-værdier for X, Y og Z og omregnes derefter til g .

Vigtig forskel: Sensorens Output Data Rate på 400 Hz betyder 400 målinger per sekund. I2C-clock på 400 kHz beskriver kommunikationsbussens bithastighed. De to tal er ikke det samme.

3D-akser og fysisk orientering – Slide 6

Et tre-akset accelerometer har et koordinatsystem, som sidder fast på selve sensoren:

- X og Y ligger i sensorchippen eller breakout-boardets plan.
- Z står vinkelret på sensorens plan.
- **Min startantagelse:** Y var op/ned og dermed den primære tapakse.
- **Den faktiske orientering i prototypen:** Y er venstre/højre, X er frem/tilbage, og Z er op/ned.

- Vender eller roterer man boardet, følger akserne med boardet; de følger ikke automatisk bordet eller rummet.
- Fortegnet viser retningen langs akse. Et tap kan give både positive og negative udsving, fordi emnet påvirkes og derefter bevæger sig tilbage.
- Firmware-rækkefølgen er stadig **X, Y, Z**, uanset hvordan boardet fysisk er monteret.

Projektets observation: Ved stilstand var det gennemsnitlige signal cirka **Z = +0,91 g**, mens taps samlet havde cirka **0,304 g** mean-centreret RMS på Z mod **0,152 g** på Y. Det var derfor den fysiske antagelse om hovedretningen, der var forkert — ikke sensorens registerrækkefølge.

Mundtligt anker til 3D-figuren: "I starten antog jeg, at Y var op/ned og derfor den primære tapakse. Gennem dataanalysen og en kontrol af sensorens fysiske orientering fandt jeg ud af, at Y i min prototype går venstre/højre, X går frem/tilbage, og Z går op/ned. Det passer med, at taps gav omtrent dobbelt så høj RMS på Z som på Y. Registerdataene var hele tiden ordnet X, Y og Z; det var min fysiske fortolkning, jeg rettede. Den endelige pipeline bruger alle tre akser plus magnitude."

Kodekort: Fra seks registerbytes til X/Y/Z – Hardware og dataindsamling

Det forkortede udsnit på slidet svarer til `readAdxlRawXYZ()` og `readSampleG()` i

`Product/firmware/src/main.cpp`:

```
uint8_t b[6];
readRegisters(addr, 0x32, b, 6);

x = (int16_t)((b[1] << 8) | b[0]);
y = (int16_t)((b[3] << 8) | b[2]);
z = (int16_t)((b[5] << 8) | b[4]);

ax = x * 0.0039f;
ay = y * 0.0039f;
az = z * 0.0039f;
```

Hvad gør koden? Den læser én synkron tredimensionel accelerationsmåling og returnerer tre værdier i enheden **g**.

Hvordan gør den? ADXL343 har seks fortløbende dataregistre fra adresse **0x32**: to bytes per akse. Den første byte i hvert par er den mindst betydende byte, og den næste er den mest betydende. Shift **<< 8** flytter high-byte otte bit mod venstre, og bitvis OR **|** samler de to bytes. Cast til **int16_t** gør resultatet til et signed 16-bit-tal, så både negativ og positiv acceleration kan repræsenteres. Til sidst omregnes råtalet med cirka **0,0039 g/LSB**.

Hvorfor læses alle seks bytes samlet? Så X, Y og Z kommer fra samme sensoropdatering og repræsenterer bevægelsen på omtrent samme tidspunkt. Det er bedre end tre helt adskilte aflæsninger, hvor signalet kan nå at ændre sig imellem dem.

Symbolerne i koden:

- **uint8_t**: ét usigneret byte, værdier fra 0 til 255.
- **int16_t**: signed 16-bit-heltal, så negative målinger er mulige.

- `<< 8`: flyt bitmønstret otte pladser til venstre.
- `|`: kombinér bittene fra high-byte og low-byte.
- `0x32`: hexadecimal registeradresse for starten af ADXL343's X/Y/Z-data.
- `0.0039f`: scale factor i `g` per LSB; `f` markerer et `float`-tal i C++.

Vigtigt hvis koden diskuteres: Koden er ikke selve machine-learning-modellen. Den skaber de fysiske inputtal, som senere gemmes i vinduet, omdannes til features og gives til modellen.

Fra rå sensordata til støjbaseline og inputsignal

Det første signalstudie bestod i at observere X-, Y- og Z-data ved stilstand og under taps. Formålet var at se sensorens normale niveau, støjens variation og størrelsen på et fysisk input, før der blev valgt tærskler og features.

I den guidede baseline holdes enheden stille i ti sekunder. De første to sekunder ignoreres, så berøring ved opstart ikke forurener idle-data eller støjestimatet. For hver måling beregnes magnitude:

$$m = \sqrt{x^2 + y^2 + z^2}$$

Systemet beregner den stationære magnitudes middelværdi μm og standardafvigelse σm . Bevægelsestærsklen sættes til:

$$\text{motion threshold} = \max(0,05 \text{ g}, 3 \times \sigma m)$$

Et inputsignal registreres, når:

$$|m - \mu m| > \text{motion threshold}$$

- μm beskriver det normale stationære niveau.
- σm beskriver støjens normale variation.
- $3 \times \sigma m$ gør tærsklen adaptiv til den målte støj.
- Minimumsværdien `0,05 g` forhindrer en alt for følsom trigger, hvis støjen er meget lille.
- Baseline-triggeren klassificerer ikke gestussen; den afgør kun, hvornår signalet tydeligt afviger fra stilstand.

Vigtig afgrænsning: Den guidede stationære baseline er en ældre capture-rutine med 20 ms polling, altså 50 Hz. Det endelige `pilot_v3`-datasæt og LIVE-modellens kontrakt bruger 400 Hz.

Mundtligt anker: "Jeg begyndte med at undersøge sensorens rå signal ved stilstand og under fysisk interaktion. Stilstand er ikke det samme som nul, fordi sensoren også måler tyngdekraft og støj. Derfor beregnede jeg en stationær magnitude-baseline og satte motion-threshold til det største af 0,05 g og tre gange støjens standardafvigelse. Det gav en datadrevet grænse mellem almindelig støj og et faktisk input."

Iterativ udvikling: fra rådata og akseantagelse til XYZ og magnitude

Efter forbindelsen mellem ADXL343 og Photon 2 var verificeret, blev de rå X-, Y- og Z-værdier observeret ved stilstand og under taps. Stilstandsdata blev brugt til at forstå tyngdekraft, offset og støj og til at etablere en stationær baseline. Derefter blev der indsamlet fem labellede optagelser per klasse: `idle`, `tap1`, `tap2`, `tap3` og `shake_lr`.

De fem optagelser per klasse udgør et **balanceret initialt datasæt**, ikke en separat støjbaseline for hver klasse. Ordet baseline bør her reserveres til den stationære støjmåling eller eventuelt en simpel første model.

Optagelserne blev kvalitetstjekket for blandt andet:

- den endelige datakontrakt på 1.600 samples og monotone timestamps;
- manglende værdier og clipping;
- tilstrækkelig fysisk bevægelse;
- klassespecifik adfærd;
- forventet impact count for `tap1`, `tap2` og `tap3`.

Capture-programmet accepterer selve serieloverførslen, hvis mindst 1.584 af de forventede 1.600 samples er modtaget, så en næsten færdig overførsel kan inspiceres. De accepterede filer, der bruges i den endelige pipeline, følger 1.600-sample-kontrakten. Det er derfor vigtigt ikke at forveksle programmets transporttolerance med modellens datakontrakt.

I en tidlig single-axis-diagnose blev det antaget, at et tap primært ville være synligt på en bestemt fysisk akse ud fra boardets vandrette placering. Dataanalysen viste, at sensorens fysiske akser ikke svarede til den intuitive retning i opstillingen. ADXL343 ændrer dog ikke dataorden til `X, Z, Y`: sensorregistre og den endelige firmware bruger fortsat den faste rækkefølge `X, Y, Z`. Fejlen var fortolkningen af, hvilken fysisk retning de navngivne akser pegede i på det monterede breakout.

Den målte data understøtter konkret skiftet fra en forventet Y-dominans til fokus på Z-responsen:

- Ved idle er det gennemsnitlige rå niveau cirka `X = 0,04 g`, `Y = 0,03 g` og `Z = 0,91 g`. Tyngdekraften ligger altså hovedsageligt på den rapporterede Z-akse i den anvendte montering.
- På tværs af `tap1`, `tap2` og `tap3` er det gennemsnitlige mean-centrede peak cirka `7,64 g` på Z mod `4,48 g` på Y.
- Tap-signalets RMS er cirka `0,304 g` på Z mod `0,152 g` på Y.
- Z er peak-dominant i 10 af de 15 endelige tap-optagelser; X er dominant i fire og Y i én.
- Ved `shake_lr` har Y derimod højere vedvarende RMS end Z. Alle tre akser indeholder derfor stadig nyttig klasseinformation.

Det er vigtigt at skelne mellem tre ting:

1. **Datastrømmens rækkefølge:** ADXL343-registerdata læses og gemmes som `X, Y, Z`.
2. **Fysisk orientering:** I den konkrete montering svarer den kraftigste normale taprespons hovedsageligt til den rapporterede Z-akse, selv om Y først blev antaget at være hovedaksen.
3. **Modelvægtning:** Den endelige featurepipeline giver de samme syv features til X, Y og Z og hardcoder ikke en større Z-vægt. StandardScaler normaliserer hver feature, hvorefter MLP'et lærer vægte ud fra, hvilke mønstre der adskiller klasserne.

Den højere Z-amplitude giver et tydeligere signalgrundlag for taps, men større rå amplitude er ikke automatisk det samme som større neural vægt efter StandardScaler. En kontrolleret feature- eller akseablation ville være nødvendig for at bevise præcis, hvor meget modelaccuracy alene skyldes Z-kanalen.

Den endelige løsning undgår derfor at afhænge af én antaget tap-akse:

- alle tre akser læses synkroniseret;

- syv features beregnes separat for X, Y og Z;
- magnitude tilføjes som en fjerde kanal;
- mean removal reducerer statistisk offset og en del af orienteringens påvirkning.

Magnitude og brugen af alle tre akser gør løsningen mindre afhængig af én bestemt akse, men gør den ikke fuldstændig rotationsinvariant. Nye monteringer og orienteringer bør stadig indgå i fremtidige trænings- og testdata.

Mundtlig udviklingshistorie: "Da jeg først tilsluttede ADXL343, undersøgte jeg de rå X-, Y- og Z-data. Ved stilstand var værdierne ikke nul på grund af tyngdekraft, offset og støj, så jeg målte først en stationær baseline. Derefter indsamlede jeg fem labellede optagelser af hver klasse og kontrollerede blandt andet signalets kvalitet og det forventede antal impacts for tap-klasserne. I de tidlige forsøg antog jeg, at tapresponsen primært ville ligge på Y, men dataanalysen viste, at Z havde næsten dobbelt så høj RMS for tap-signalerne. Det betød, at min fortolkning af sensorens fysiske orientering var forkert, selv om datastrømmen stadig var ordnet som X, Y og Z. Jeg flyttede derfor fokus væk fra den tidlige Y-antagelse og anvendte i den endelige pipeline alle tre akser samt magnitude. Det gav modellen adgang til den kraftige Z-respons uden at kassere information fra X og Y. Resultaterne blev bedre, men uden en kontrolleret akseablation kan jeg ikke tilskrive hele forbedringen til Z alene."

Sensoropløsning, måleområde og enheder

En sensors måleområde angiver de mindste og største fysiske værdier, den kan repræsentere. Opløsning beskriver det mindste trin, der kan skelnes. Et større måleområde reducerer risikoen for clipping, men kan give grovere effektiv opløsning afhængigt af sensoren.

- Acceleration angives ofte i **g**, hvor cirka **1 g** svarer til tyngdeaccelerationen.
- Clipping eller saturation opstår, hvis den fysiske værdi ligger uden for sensorens valgte område.
- Rå heltalsmålinger skal omregnes med sensorens scale factor.
- Sensorstøj er variation i målingen, som ikke skyldes den ønskede bevægelse.
- Måleområde, opløsning, samplingfrekvens og enheder bør gemmes som metadata.

I mit projekt: ADXL343 anvendes i ± 16 g full-resolution og konverteres til acceleration i **g**. Området er valgt for at kunne rumme hurtige taps uden saturation.

Sampling og samplingfrekvens

400 Hz betyder 400 samples per sekund:

$$\text{sampleinterval} = 1 / 400 \text{ s} = 0,0025 \text{ s} = 2,5 \text{ ms}$$

Fire sekunder giver:

$$400 \text{ samples/s} \times 4 \text{ s} = 1.600 \text{ samples}$$

Hvorfor 400 Hz?

De første forsøg ved 50 Hz havde 20 ms mellem samples og kunne overse dele af hurtige taps. Ved 400 Hz er der 2,5 ms mellem samples, hvilket giver bedre tidslig opløsning. Ulemperne er mere data, større buffer og potentielt mere højfrekvent støj.

Nyquist-frekvens og aliasing

Når et kontinuerligt signal samples, kan frekvenser over halvdelen af samplingfrekvensen ikke repræsenteres entydigt. Halvdelen af samplingfrekvensen kaldes Nyquist-frekvensen. Aliasing betyder, at for hurtige signaldele fejlagtigt ser ud som lavere frekvenser i de samplede data.

- Ved 400 Hz er Nyquist-frekvensen 200 Hz.
- Samplingfrekvensen bør være mere end dobbelt så høj som den højeste relevante signalfrekvens.
- Et analogt eller digitalt low-pass-filter kan dæmpe uønskede høje frekvenser.
- En høj samplingfrekvens giver flere detaljer, men øger data-, hukommelses- og energibehovet.
- Projektets valg af 400 Hz blev fundet empirisk; der blev ikke udført en fuld frekvensanalyse af gestusserne.

Filtrering

Filtrering bruges til at fremhæve relevante dele af et signal eller dæmpe støj. Et low-pass-filter dæmper høje frekvenser, et high-pass-filter dæmper lave frekvenser, og et band-pass-filter beholder et bestemt frekvensområde.

- Filtrering er preprocessing og kan ændre de informationer, modellen ser.
- Et anti-aliasing-filter anvendes før eller som del af samplingkæden for at dæmpe frekvenser over Nyquist-grænsen.
- Kraftig filtrering kan fjerne korte taps, som netop er relevante i dette projekt.
- Filtrervalg bør baseres på signalets frekvensindhold og ikke kun på intuition.

I mit projekt: Der anvendes mean removal og engineered features, men ikke et egentligt udviklet digitalt low-pass- eller band-pass-filter i modelpipen.

I2C

I2C er en seriel bus med en datalinje, SDA, og en clocklinje, SCL. Enheder identificeres med adresser. Firmwaren kontrollerer sensorens adresse og device-ID ved opstart.

6. Datasættet, labels og klassebalance

Hurtigt svar

Det deployede datasæt har 25 accepterede optagelser: fem optagelser for hver af fem klasser. Alle kommer fra én person og én afsluttende session. Hver optagelse varer fire sekunder og har 1.600 samples.

Klasser

- `idle`
- `tap1`
- `tap2`
- `tap3`
- `shake_1r`

Label og features

- **Label:** Det korrekte facit, eksempelvis `tap2`.
- **Feature:** En målbar numerisk repræsentation af en relevant egenskab ved inputtet, som modellen bruger som input til sin beslutning.

En feature er ikke nødvendigvis symbolsk og bestemmer ikke alene modellens output. Modellen lærer vægte, som afgør, hvordan alle features kombineres. I dette projekt er features hånddesignede statistiske beskrivelser af tidsserien, eksempelvis signalets variation, yderpunkter, energi, peaks og hurtigste ændring.

Færre features gør ikke datasættet mindre i betydningen færre træningseksempler. Det reducerer **inputdimensionen**: Hver optagelse repræsenteres med 28 tal i stedet for tusindvis af samples. Det kan reducere modelstørrelse, RAM-behov og beregningsarbejde på en ressourcebegrænset enhed.

Mundtligt anker: "En feature er en numerisk egenskab, der er udledt af inputdata og givet til modellen. Den fortæller ikke alene, hvilket output der skal vælges; modellen lærer, hvordan flere features skal vægtes og kombineres. Jeg bruger bevidst et lille featureinput for at reducere inputdimension, modelstørrelse og beregning på Photon 2."

Metadata

Metadata er information om dataene, som ikke nødvendigvis er selve sensorsignalet, men som er nødvendig for at forstå, reproducere og kvalitetssikre det.

- Eksempler er label, samplingfrekvens, enhed, sensorområde, tidspunkt, bruger, session og optagebetingelser.
- Manglende metadata gør det svært at opdage forskelle mellem sessioner.
- Metadata kan bruges til at lave mere realistiske datasplit, eksempelvis efter person eller session.
- Metadata må ikke ukritisk bruges som modelfeatures, hvis de afslører labelen på en kunstig måde.

I mit projekt: CSV og tilhørende JSON beskriver blandt andet label, tempo, kraft, session og kvalitetskontrol.

Dataset bias og repræsentativitet

Dataset bias opstår, når de indsamlede data systematisk ikke repræsenterer den virkelighed, modellen skal bruges i. En model kan kun lære den variation, som findes i dens data.

- Flere samples er ikke nok, hvis de alle kommer fra samme person og situation.
- Variation i bruger, montering, hastighed, kraft og optagedag kan være vigtig.
- Meget streng kvalitetsfiltrering kan skabe unaturligt rene data.
- Et balanceret datasæt løser klasseubalance, men ikke andre former for bias.

I mit projekt: Den største bias er én operatør og én afsluttende session.

Hvorfor balancerede klasser?

Der er lige mange optagelser fra hver klasse. Det reducerer risikoen for, at modellen favoriserer en klasse alene, fordi den forekommer oftere.

Kvalitetskontrol

Optagelser kunne afvises ved blandt andet forkert antal taps, for meget bevægelse under idle eller utilstrækkelig shake. Afvisning forbedrer labelkvaliteten, men kan også gøre datasættet mindre repræsentativt, hvis kun meget "rene" gestusser accepteres.

De automatiske gates i `randomized_capture_gui.py` er:

- **Alle klasser:** afvisning ved mulig clipping, når en rå akse når mindst `15,5 g`.
- **Idle:** accepteres kun, når dynamisk peak-magnitude er højst `0,15 g`.
- **Tap:** største akseudslag skal være mindst `0,40 g`, og det registrerede antal impact-events skal svare til labelen.
- **Impact-event:** dynamisk magnitude skal være mindst `0,35 g`; aktive områder med mere end `150 ms` mellemrum tælles som separate events.
- **Shake:** dynamisk RMS skal være mindst `0,10 g`, og bevægelsen skal strække sig over mindst `300 ms`.

Disse værdier er heuristiske kvalitetsgrænser til dataindsamlingen. De er ikke MLP'ens lærte beslutningsgrænser og må heller ikke forveksles med LIVE-inferencens confidence threshold på `0,75`.

Største begrænsning

25 optagelser er meget lidt. De kommer desuden fra samme person og session. Modellen kan derfor have lært netop denne persons og denne sessions mønstre frem for generelle gestusmønstre.

Rapportens snapshot kontra den aktuelle datapulje

Rapportens model, confusion matrix og metrics tilhører det autoritative, balancerede snapshot i `pilot_v3/20260810_141717/accepted`: 25 optagelser, fem per klasse. `config.yaml` peger fortsat kun på denne mappe, så det er disse 25 optagelser, den deployede model er trænet og evalueret på.

På tværs af alle `pilot_v3`-sessioner findes der aktuelt 54 CSV-filer markeret som accepterede. Fem tidlige filer har kun 1.200 samples og følger derfor ikke den endelige kontrakt på 1.600 samples. Der er således 49 kontraktkompatible, accepterede optagelser til rådighed som en udvidet kandidatpulje:

- `idle`: 5
- `shake_lr`: 5
- `tap1`: 5
- `tap2`: 14
- `tap3`: 20

De 24 ekstra kontraktkompatible optagelser ligger kun i `tap2` og `tap3`, så den udvidede pulje er ikke klassebalanceret. De bør omtales som **yderligere indsamlede kandidatdata**, ikke som den rapporterede models træningsdata, før datasættet er kurateret, et nyt split er defineret, og modellen er retrænet og evalueret igen.

Hvis nye post-submission-optagelser ligger uden for repository, skal antal, klassefordeling, bruger/session og kvalitetsstatus dokumenteres, før tallet på præsentationen opdateres.

Mundtligt anker: "Rapportens dokumenterede resultater er et snapshot baseret på 25 balancerede optagelser. Jeg har fortsat dataarbejdet, og repository indeholder nu en større pulje af kontraktkompatible optagelser. De ekstra data er endnu ikke balancerede eller indarbejdet i den rapporterede model, så jeg adskiller tydeligt det afleverede resultat fra den videre udvikling. Næste trin er kuratering, balancering, retræning og en ny evaluering."

7. Preprocessing og features

Preprocessing

Preprocessing er de transformationer, der udføres på data, før modellen bruger dem.

Fjernelse af gennemsnit

For hver akse trækkes vinduets gennemsnit fra hvert sample:

`centreret værdi = oprindelig værdi - vinduets gennemsnit`

Det reducerer statisk offset og påvirkningen fra enhedens orientering. Det kan samtidig fjerne information om absolut orientering, så det er et designvalg og ikke en universelt korrekt løsning.

Magnitude

`magnitude =  $\sqrt{x^2 + y^2 + z^2}$`

Magnitude beskriver samlet bevægelsesstyrke på tværs af akserne og er mindre afhængig af, hvilken enkelt akse bevægelsen rammer.

Feature engineering

Fire kanaler: X, Y, Z og magnitude.

Syv features per kanal:

1. standardafvigelse;
2. minimum;
3. maksimum;
4. range;
5. energi;
6. peak count;
7. maximum absolute difference.

`4 kanaler × 7 features = 28 features`

De syv features beskriver forskellige egenskaber ved én kanal:

1. **Standardafvigelse:** Hvor meget signalet varierer omkring sit gennemsnit.
2. **Minimum:** Det største negative udsving efter centrering.
3. **Maksimum:** Det største positive udsving efter centrering.
4. **Range:** `maksimum - minimum`; signalets samlede spændvidde.
5. **Energi:** Gennemsnittet af de kvadrerede samples, `mean(x2)`; et mål for samlet signalstyrke, ikke fysisk energi i joule.
6. **Peak count:** Antal tydelige lokale peaks i det absolutte signal. Implementationen bruger mindst `0,05 g` og mindst otte samples, svarende til 20 ms ved 400 Hz, mellem registrerede peaks.
7. **Maximum absolute difference:** Den største værdi af `|xi - xi-1|`; en indikator for den hurtigste ændring mellem to samples. Det er ikke helt det samme som fysisk jerk, fordi værdien ikke divideres med tidsforskellen.

Featurevektorens orden er X-features, Y-features, Z-features og til sidst magnitude-features. Python og C++ skal bruge præcis denne rækkefølge.

Kodekort: Sådan bliver fire kanaler til 28 features – Slides 10–11

Det centrale princip i `extractFeatures()` er:

```

for (size_t c = 0; c < 4; ++c) {
    for (size_t i = 0; i < 1600; ++i) {
        float ax = window[i].ax;
        float ay = window[i].ay;
        float az = window[i].az;
        if (c == 0) ch[i] = ax;
        else if (c == 1) ch[i] = ay;
        else if (c == 2) ch[i] = az;
        else ch[i] = sqrtf(ax*ax + ay*ay + az*az);
    }
    for (size_t i = 0; i < 1600; ++i)
        ch[i] -= mean;
    channelFeatures(ch, 1600, out7);
    for (size_t j = 0; j < 7; ++j)
        features[f++] = out7[j];
}

```

Udsnittet er forkortet for læsbarhed; i den faktiske kode ligger valg af kanal, beregning af gennemsnit og kopiering i hver sin løkke.

Hvad gør koden? Den bygger modellens inputvektor på 28 tal.

Hvordan gør den? Den gennemløber fire kanaler: X, Y, Z og magnitude. Hver kanal mean-centreres, `channelFeatures()` beregner syv statistikker, og `features[f++]` lægger dem efter hinanden i den aftalte rækkefølge.

Hvorfor? $1.600 \times 3 = 4.800$ rå akseværdier er en stor og tidsafhængig inputrepræsentation. De 28 features komprimerer signalet til variation, udsving, styrke, peaks og hurtige ændringer, så et lille MLP kan køre effektivt på Photon 2.

Den vigtigste kodekontrakt: Python-træningen og C++-firmwaren skal beregne de samme 28 features i samme orden. Hvis eksempelvis Y- og Z-grupperne byttes, modtager modellen andre egenskaber på de pladser, dens vægte er trænet til.

Hvorfor feature engineering?

Det komprimerer tusindvis af rå målinger til få forklarlige tal. Det reducerer modellens størrelse og beregningsbehov, men kan kassere tidslig information, som en model på rå tidsserier kunne have udnyttet.

Mundtligt anker: "Fire sekunder giver 1.600 samples på hver fysisk akse. Jeg beregner også magnitude og udtrækker syv statistiske egenskaber fra hver af de fire kanaler. Det giver 28 inputs, som beskriver styrke, variation, peaks og hurtige ændringer. Det er en bevidst komprimering, der gør modellen mindre og mere forklarlig, men som også mister noget af den præcise tidslige rækkefølge."

Tidsdomæne og frekvensdomæne

Tidsdomænet beskriver, hvordan signalets værdi ændrer sig over tid. Frekvensdomænet beskriver, hvilke gentagelseshastigheder eller frekvenskomponenter signalet består af.

- Minimum, maksimum, energi og peak count er tidsdomænefeatures.
- En Fourier-transformation kan flytte analysen til frekvensdomænet.
- Frekvensfeatures kan være nyttige ved periodiske vibrationer eller lyd.
- Taps er korte transienter, mens en shake kan have mere periodisk struktur.
- Valg af domæne afhænger af, hvilke forskelle mellem klasserne man ønsker at fremhæve.

I mit projekt: De 28 features beregnes i tidsdomænet; FFT- eller spektrale features anvendes ikke.

Feature selection

Feature selection betyder at vælge de mest informative inputfeatures og fjerne irrelevante eller stærkt redundante features.

- Færre features kan reducere beregning, hukommelse og overfitting.
- Features kan vurderes med domæneviden, visualisering, statistiske metoder eller modelbaseret importance.
- Feature selection skal udføres uden at lade testsættet påvirke valget.
- Feature engineering skaber features; feature selection vælger mellem dem.

I mit projekt: Der bruges et fast sæt på 28 features. Der blev ikke gennemført en systematisk feature-ablation, hvor features fjernes én ad gangen og effekten måles.

8. StandardScaler

Hurtigt svar

StandardScaler bringer features over på sammenlignelige skalaer, så numerisk store features ikke automatisk får størst indflydelse.

Formel

$$z = (x - \mu) / \sigma$$

- x : featureværdien.
- μ : featureens gennemsnit i træningsdata.
- σ : featureens standardafvigelse i træningsdata.
- z : den standardiserede værdi.

Efter standardisering vil træningsdata typisk have middelværdi omkring 0 og standardafvigelse omkring 1 for hver feature.

Data leakage

Scalerens μ og σ må kun beregnes fra træningsdata. Hvis testdata påvirker dem, opstår data leakage. De lærte værdier eksporteres til C++, så enheden bruger præcis samme transformation.

Hvordan skaleringen bruges i projektet

For hver af de 28 featuredimensioner beregner `StandardScaler` sit eget gennemsnit og sin egen standardafvigelse på træningssættet. En `peak_count`-værdi skaleres derfor med statistikken for `peak_count`, mens en energiværdi skaleres med energifeaturens egen statistik.

Eksempel: Hvis en feature har træningsgennemsnit 8, standardafvigelse 2 og den nye værdi er 12, bliver den skalerede værdi $(12 - 8) / 2 = 2$. Den ligger dermed to standardafvigelser over træningsgennemsnittet.

De 28 middelværdier og 28 scale-værdier eksporteres som konstante arrays til firmwaren. Photon 2 fitter ikke en ny scaler, men genbruger træningssættets værdier. Hvis en scale-værdi er ekstremt tæt på nul, bruger C++-koden 1,0 som divisor for at undgå division med nul.

Mean centering er ikke StandardScaler

- **Mean centering:** Udføres på de 1.600 samples i hvert enkelt vindue før feature extraction og reducerer vinduets statiske offset.
- **StandardScaler:** Udføres på de 28 færdige features og bruger statistik fra hele træningssættet til at gøre featuredimensionerne sammenlignelige.

StandardScaler gør ikke nødvendigvis data normalfordelte. Den centrerer hver feature omkring nul og giver den typisk standardafvigelse omkring én i træningsdataene.

Mundtligt anker: "De 28 features har forskellige numeriske størrelsesordener. Derfor trækker jeg træningsgennemsnittet fra hver feature og dividerer med dens standardafvigelse. Scaleren fittes kun på træningsdata for at undgå data leakage, og de samme værdier eksporteres til Photon 2, så preprocessing er identisk under træning og inferens."

MLP – Multilayer Perceptron

Et MLP er en type feedforward neuralt netværk. "Feedforward" betyder, at informationen bevæger sig i én retning: fra inputlaget gennem de skjulte lag og videre til outputlaget. Netværket lærer at kombinere inputtene ved hjælp af vægte og biases.

- Et MLP kan lære ikke-lineære sammenhænge.
- Inputlaget modtager de værdier, modellen skal arbejde med.
- De skjulte lag kombinerer værdierne og finder mønstre.
- Outputlaget giver én score for hver mulig klasse.

- Flere neuroner og lag giver større modelkapacitet, men også større risiko for overfitting og højere ressourceforbrug.

I mit projekt: MLP'et har strukturen $28 \rightarrow 32 \rightarrow 16 \rightarrow 5$. Det modtager 28 features, har to skjulte lag og producerer fem klassescores.

Neuron, vægte og bias

En kunstig neuron er en lille matematisk beregning. Den modtager flere input, ganger hvert input med en vægt, lægger resultaterne sammen og tilføjer en bias. Resultatet sendes normalt gennem en aktiveringsfunktion.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

$$a = f(z)$$

- x er inputværdierne.
- w er vægtene, som afgør hvor meget hvert input betyder.
- b er bias, som forskyder neuronens resultat.
- Vægte og biases er parametre, modellen lærer under træningen.
- En neuron er ikke en fysisk node; den er en beregning i software.

Enkelt eksempel: Hvis en neuron modtager "energi" og "peak count", kan træningen give de to features forskellige vægte afhængigt af, hvor nyttige de er til at skelne gestusser.

I modellen har det første skjulte lag 32 neuroner, og hvert neuron modtager alle 28 features. Det næste lag har 16 neuroner, og outputlaget har fem neuroner. Det giver:

- Første lag: 28×32 vægte + 32 biases = 928 parametre.
- Andet lag: 32×16 vægte + 16 biases = 528 parametre.
- Outputlag: 16×5 vægte + 5 biases = 85 parametre.
- I alt: 1.541 lærte modelparametre.

Det er antallet af vægte og biases, ikke antallet af neuroner.

Mundtligt anker: "Et neuron er en matematisk funktion. Det vægter sine inputs, lægger en bias til og bruger en aktiveringsfunktion. Vægtene bestemmer, hvor stærkt forskellige input påvirker neuronet, og de læres under træningen. Flere neuroner kan derfor lære forskellige kombinationer af de 28 features."

Aktiveringsfunktion

En aktiveringsfunktion bestemmer, hvordan en neurons beregnede værdi sendes videre. Uden ikke-lineære aktiveringsfunktioner ville flere lag stadig samlet set opføre sig som én lineær transformation, og netværket kunne ikke lære komplekse beslutningsgrænser.

- Aktiveringsfunktionen anvendes efter den vægtede sum.
- Den gør det muligt for netværket at beskrive ikke-lineære mønstre.

- Forskellige aktiveringsfunktioner har forskellige egenskaber.

I mit projekt: De skjulte lag bruger ReLU, mens outputtet omdannes med softmax.

ReLU

ReLU betyder Rectified Linear Unit. Funktionen sender positive værdier videre uændret og erstatter negative værdier med nul.

$$\text{ReLU}(x) = \max(0, x)$$

- Hvis input er 4, bliver output 4.
- Hvis input er -4, bliver output 0.
- ReLU er enkel og billig at beregne.
- ReLU tilfører den ikke-linearitet, som gør de skjulte lag nyttige.
- En mulig ulempe er, at en neuron kan ende med altid at give nul; det kaldes nogle gange en "dead ReLU".

I mit projekt: ReLU anvendes efter begge skjulte lag med henholdsvis 32 og 16 neuroner.

Logits

Logits er modellens rå outputværdier før softmax. De behøver ikke ligge mellem 0 og 1 og behøver ikke summere til noget bestemt.

- Hver klasse har én logit.
- En større logit betyder, at modellen foretrækker den klasse relativt til de andre.
- Softmax omdanner logits til sammenlignelige positive værdier.

I mit projekt: Outputlaget producerer fem logits, én for hver gestusklasse.

Softmax

Softmax omdanner modellens rå logits til fem positive scores, som tilsammen giver 1. Det gør det muligt at sammenligne klassernes relative sandsynlighed eller confidence.

$$\text{softmax}(z_i) = \exp(z_i) / \sum \exp(z_j)$$

- Eksponentialfunktionen gør alle værdier positive.
- Division med summen får værdierne til at summere til 1.
- Klassen med den største softmax-værdi vælges normalt som prediction.
- En høj softmax-score er ikke garanti for, at modellen har ret.
- Softmax-output kan være dårligt kalibreret, især med få træningsdata.

I mit projekt: Den højeste af fem softmax-scores er modellens klasseforslag. Forslaget skal også passere en confidence-grænse og beslutningslogikken, før en kommando udføres.

Forward-pass

Et forward-pass er den beregning, der sender ét input gennem hele det trænede netværk for at producere et output. Under inference udføres kun forward-pass; modellen lærer ikke nye vægte på enheden.

- Input skales først.
- Hvert lag beregner vægtet sum, bias og aktivering.
- Outputlaget producerer logits.
- Softmax producerer klassescores.

I mit projekt: Forward-pass er implementeret i C++ på Photon 2 med de vægte og biases, der blev lært i Python.

Den konkrete rækkefølge er:

1. De 28 features standardiseres.
2. Første lag beregner $z_1 = W_1x + b_1$, hvorefter $a_1 = \text{ReLU}(z_1)$ giver 32 aktiveringer.
3. Andet lag beregner $z_2 = W_2a_1 + b_2$, hvorefter $a_2 = \text{ReLU}(z_2)$ giver 16 aktiveringer.
4. Outputlaget beregner $z_3 = W_3a_2 + b_3$ og producerer fem logits.
5. Softmax omdanner de fem logits til positive klassescores, der summerer til 1.
6. Klassen med den største score bliver modellens kandidat.

Softmax-implementationen trækker først den største logit fra alle logits, før eksponentialfunktionen beregnes. Det ændrer ikke den matematiske fordeling, men reducerer risikoen for numerisk overflow.

Mundtligt anker: "Et forward-pass sender ét featureinput frem gennem den færdigtrænede model. Hvert lag beregner vægtede summer og biases, de skjulte lag anvender ReLU, og outputlaget anvender softmax. Der foregår ingen læring eller backpropagation på Photon 2; de eksporterede vægte bliver kun brugt."

Loss-funktion

En loss-funktion giver et tal for, hvor forkert modellens prediction er i forhold til den korrekte label. Træningens mål er at justere modellens parametre, så loss bliver mindre.

- En korrekt og sikker prediction giver typisk lav loss.
- En forkert og sikker prediction giver typisk høj loss.
- Loss bruges til træning; den er ikke det samme som accuracy.
- Ved flerklassifikation bruges ofte cross-entropy loss.

I mit projekt: `MLPClassifier` håndterer loss-beregningen som en del af træningen. Firmwaren beregner ikke loss, fordi den kun udfører inference.

Backpropagation

Backpropagation er algoritmen, der beregner, hvor meget hver vægt og bias bidrog til modellens fejl. Beregningen starter ved outputtet og bevæger sig baglæns gennem netværket ved hjælp af den matematiske kæderegul.

- Først udføres et forward-pass.
- Loss-funktionen måler fejlen.
- Backpropagation beregner gradienten for hver parameter.
- En gradient beskriver, hvordan en lille ændring af parameteren påvirker loss.
- Backpropagation ændrer ikke selv vægtene; den beregner den information, optimeren bruger til at ændre dem.
- Processen gentages over mange træningseksempler.

I mit projekt: Scikit-learns `MLPClassifier` udfører backpropagation under træningen i Python. Der foregår ingen backpropagation på Photon 2.

Gradient descent og optimizer

Gradient descent er princippet om at ændre modellens parametre i den retning, der reducerer loss. Optimizeren bruger gradienterne fra backpropagation til at beregne de konkrete opdateringer.

$$\text{ny vægt} = \text{gammel vægt} - \text{learning rate} \times \text{gradient}$$

- Gradientens retning viser, hvordan loss vokser mest.
- Derfor bevæger man sig i den modsatte retning.
- Learning rate bestemmer størrelsen på hvert skridt.
- For høj learning rate kan springe forbi en god løsning.
- For lav learning rate kan gøre træningen meget langsom.

I mit projekt: Learning rate er sat til `0,001`. Det er en hyperparameter, ikke en værdi modellen selv lærer.

Epoch, batch og iteration

En epoch betyder, at træningsalgoritmen har haft mulighed for at gennemgå hele træningssættet én gang. En batch er den gruppe træningseksempler, der behandles sammen før en parameteropdatering. En iteration er normalt én behandling af en batch.

- Flere epochs giver modellen flere muligheder for at lære, men kan øge overfitting.
- Små batches giver hyppigere og mere støjende opdateringer.
- Store batches kræver mere hukommelse.
- `max_iter` i scikit-learns `MLPClassifier` er en øvre grænse for træningsiterationer/epochs afhængigt af solverens arbejdsform.

- At træningen stopper ved grænsen betyder ikke nødvendigvis, at den er konvergeret.

I mit projekt: Den konfigurerede `batch_size` er 32, learning rate er 0,001, og `max_iter` er 500. Træningssættet har kun 20 optagelser, så scikit-learn begrænser i praksis batchen til størrelsen på træningssættet.

Konvergens

Konvergens betyder, at optimeringen nærmer sig en stabil løsning, hvor loss ikke længere forbedres væsentligt. Manglende konvergens kan betyde, at modellen behøver flere iterationer, en anden learning rate, bedre skalering eller en anden model.

- Lavere training loss er ikke automatisk bedre generalisering.
- Man bør sammenligne trænings- og valideringsadfærd.
- En convergence warning er information om træningsprocessen, ikke i sig selv bevis på en ubrugelig model.

I mit projekt: Grænsen blev hævet fra 40 til 500, fordi 40 iterationer stoppede før konvergens på det endelige femklassedatasæt.

Decision boundary

En decision boundary er grænsen i feature-rummet, hvor modellen skifter fra at foretrække én klasse til en anden. Med 28 features ligger grænsen i et 28-dimensionelt rum og kan derfor ikke visualiseres direkte på en almindelig todimensional graf.

- En lineær model bruger lineære grænser.
- Et MLP med ikke-lineære aktiveringer kan lære mere komplekse grænser.
- En mere kompleks grænse kan passe data bedre, men også overfitte.

I mit projekt: MLP'et lærer grænser mellem de fem gestusklasser ud fra kombinationer af de 28 features.

Random forest og beslutningstræer

Et beslutningstræ laver en række if/else-lignende opdelinger af feature-rummet. En random forest kombinerer mange træer, som er trænet med variation i data og features, og afgør normalt klassifikationen ved afstemning.

- Træer kræver normalt ikke StandardScaler.
- De kan modellere ikke-lineære relationer.
- Et enkelt træ er let at fortolke, men kan overfitte.
- En forest er ofte mere robust, men fylder mere end ét træ.
- Træbaserede modeller kan konverteres til C og bruges på microcontrollere med værktøjer som emlearn.

I mit projekt: Træningskoden understøtter en random-forest-konfiguration, men den deployede model er et MLP.

Deep learning

Deep learning bruger neurale netværk med flere lag til at lære repræsentationer direkte eller næsten direkte fra data. Det kan reducere behovet for hånddesignede features, men kræver ofte mere data og flere ressourcer.

- Deep learning er særligt stærkt til billeder, lyd og komplekse tidsserier.
- Mere kompleksitet er ikke automatisk bedre på små datasæt.
- Embedded deployment kan kræve kvantisering eller specialiserede biblioteker.
- Træning udføres ofte på en kraftig computer, mens kun inference deployeres.

I mit projekt: MLP'et er et lille neuralt netværk på engineered features. Det er langt mindre end typiske deep-learning-modeller på rå data.

Convolution

En convolution anvender et lille filter eller en kernel på lokale områder af et signal. De samme kernel-vægte genbruges på tværs af signalet, så modellen kan genkende det samme lokale mønster forskellige steder.

- For en 1D-tidsserie bevæger kernelen sig langs tidsaksen.
- Weight sharing reducerer antallet af parametre sammenlignet med fuldt forbundne lag.
- Convolution kan lære lokale mønstre som kanter i billeder eller impulser i sensorsignaler.
- Kernel size bestemmer, hvor stort et lokalt område laget ser ad gangen.
- Flere convolution-lag kan kombinere simple lokale mønstre til mere komplekse mønstre.

1D-CNN

En 1D-CNN er et convolutional neural network til sekventielle data som lyd, vibrationer eller accelerometertidsserier. Den kan lære tidslige features direkte fra rå eller let behandlede samples.

- Den bevarer mere tidslig struktur end en model på kun globale statistikfeatures.
- Den kan være mere beregnings- og hukommelseskrævende.
- Den kræver normalt mere og mere varieret træningsdata.
- Inputformen og preprocessing skal stadig være identisk under træning og deployment.

I mit projekt: En 1D-CNN var en mulig kandidat, men featurebaseret MLP blev valgt på grund af det lille datasæt, forklarlighed og enklere C++-deployment.

10. Træning, test og generalisering

Supervised learning

Modellen trænes på eksempler, hvor den korrekte klasse allerede er kendt.

Unsupervised learning

Ved unsupervised learning har modellen ikke korrekte labels for hvert eksempel. Den forsøger i stedet at finde struktur, grupper eller afvigelser i dataene.

- Clustering er et eksempel på unsupervised learning.
- Det kan bruges til udforskning eller anomaly detection.
- Fundne grupper er ikke automatisk meningsfulde klasser; de skal fortolkes.
- Supervised og unsupervised beskriver træningssignalet, ikke om algoritmen er "simpel" eller "intelligent".

I mit projekt: Gestusmodellen er supervised, fordi alle optagelser har en kendt label.

Train, validation og test

Træningsdata

Bruges til at lære scalerens værdier og modellens vægte.

Testdata

Holdes ude af træningen og bruges til en afsluttende vurdering på usete eksempler.

Validation

Bruges normalt under modeludvikling til valg af model og hyperparametre. Et testsæt bør ikke bruges gentagne gange til at træffe udviklingsvalg, fordi det så indirekte bliver en del af udviklingen.

Stratificeret 80/20 holdout

Et holdout-split opdeler datasættet én gang. Projektets 25 optagelser deles i:

- 80 % træning = 20 optagelser.
- 20 % test = 5 optagelser.

Splittet er stratificeret. Det betyder, at klassernes fordeling bevares. Da der er fem optagelser fra hver af fem klasser, indeholder træningssættet fire optagelser per klasse, mens testsættet indeholder én optagelse per klasse.

Hvorfor: Træningssættet bruges til at fitte scaler og model. Testsættet holdes ude af træningen og bruges som en afsluttende kontrol på usete optagelser. Stratificering forhindrer, at en klasse ved et tilfælde mangler i trænings- eller testdelen.

Begrænsning: Testsættet har kun fem eksempler. Én fejl ændrer accuracy med $1 / 5 = 20$ procentpoint, så 80 % er ikke et præcist estimat for fremtidig generalisering.

Mundtligt anker: "Jeg anvender et stratificeret 80/20 holdout. Det giver 20 træningsoptagelser og fem testoptagelser, præcis én testoptagelse fra hver klasse. Testen er let at forklare, men meget følsom, fordi én fejl ændrer accuracy med 20 procentpoint."

Overfitting og generalisering

Overfitting

Overfitting betyder, at modellen lærer træningsdata meget specifikt, men klarer sig dårligt på nye data. Risikoen er høj her på grund af det lille datasæt og den samme bruger/session.

Generalisering

Generalisering er evnen til at fungere på nye data fra den virkelige målgruppe og anvendelse. Projektet dokumenterer funktionalitet, men ikke brugeruafhængig generalisering.

Hyperparametre og parametre

- **Hyperparametre:** Valgt før træning, eksempelvis lagstørrelser, learning rate og maksimalt antal iterationer.
- **Lærte parametre:** Vægte, biases og scalerens værdier, som bestemmes ud fra træningsdata.

11. Resultater og metrics

Projektets resultater

- Held-out accuracy: 80 %, altså 4 af 5 testoptagelser.
- Macro precision: 70,0 %.
- Macro recall: 80,0 %.
- Macro F1: 73,3 %.
- Femfolds-cross-validation: 76 %.
- `tap1` blev forvekslet med `tap2` i det valgte test-split.
- Inference: cirka 345 μ s i gennemsnit og 364 μ s maksimalt i den registrerede LIVE-test.
- Firmware: 27.950 B flash og 46.686 B RAM.

Confusion matrix

Rækker viser normalt de sande klasser, og kolonner viser de forudsagte klasser. Diagonalen er korrekte predictions. Tal uden for diagonalen viser konkrete forvekslinger.

True positive, false positive og false negative

Når én klasse vurderes ad gangen, kan predictionerne opdeles sådan:

- **True positive, TP:** Modellen forudsiger klassen, og det er korrekt.
- **False positive, FP:** Modellen forudsiger klassen, men den sande klasse er en anden.
- **False negative, FN:** Eksemplet tilhører klassen, men modellen vælger en anden.
- **True negative, TN:** Eksemplet tilhører ikke klassen, og modellen vælger den heller ikke.

I flerklassifikation beregnes disse størrelser typisk separat for hver klasse ved at betragte den valgte klasse mod alle de andre.

Accuracy

$$\text{accuracy} = \text{antal korrekte predictions} / \text{antal predictions}$$

Accuracy er let at forstå, men kan være misvisende ved ubalancerede klasser.

Precision

$$\text{precision} = \text{TP} / (\text{TP} + \text{FP})$$

Af alle de eksempler, modellen kaldte en bestemt klasse, hvor mange var faktisk den klasse?

Recall

$$\text{recall} = \text{TP} / (\text{TP} + \text{FN})$$

Af alle de virkelige eksempler fra en klasse, hvor mange fandt modellen?

F1-score

$$\text{F1} = 2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$$

F1 er det harmoniske gennemsnit af precision og recall og bliver lav, hvis en af dem er lav.

Macro average

Metriken beregnes separat for hver klasse, og klasserne får derefter lige stor vægt i gennemsnittet.

Cross-validation (CV) – fem testrunder med skiftende testdata

Helt enkelt: Forestil dig de 25 optagelser fordelt i fem bunker med fem optagelser i hver. Hver bunke indeholder én optagelse fra hver klasse. Modellen trænes og testes fem gange:

1. Træn på bunke 2–5, og test på bunke 1.
2. Træn på bunke 1, 3, 4 og 5, og test på bunke 2.
3. Fortsæt, indtil alle fem bunker har været testdata én gang.

I hver runde er der derfor 20 træningsoptagelser og fem testoptagelser. Til sidst beregnes gennemsnittet af de fem testresultater. "5-fold" betyder fem bunker, "cross-validation" betyder den gentagne evaluering, og "stratificeret" betyder, at alle klasser er repræsenteret i hver bunke.

Data opdeles flere gange, så hver del bruges som validering én gang. Det giver flere målinger end ét enkelt split, men femfolds-cross-validation på kun 25 optagelser er stadig baseret på meget få og tæt relaterede eksempler.

Projektets 76 % kan reproducere med `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`. De fem fold-accuracies er 60 %, 60 %, 100 %, 80 % og 80 %. I hver fold skal en ny StandardScaler og en ny MLP kun fittes på foldens træningsdel.

- Stratificering sikrer én optagelse fra hver klasse i hver testfold.
- Shuffle og seed 42 gør den valgte opdeling reproducerbar.
- Gennemsnittet er $(0,60 + 0,60 + 1,00 + 0,80 + 0,80) / 5 = 0,76$.
- Cross-validation skaber ikke nye personer eller sessioner.
- Den nuværende `train.py` udskriver kun det faste held-out-split; CV-resultatet kræver en separat kontrolberegning.

I femfolds-cross-validation opdeles datasættet i fem folds med fem optagelser i hver. Modellen trænes fem gange. I hver runde bruges fire folds, altså 20 optagelser, til træning og det resterende fold med fem optagelser til validering. Hver optagelse bliver dermed brugt som valideringsdata præcis én gang.

En ny StandardScaler og en ny MLP skal fittes inde i hver runde og kun på rundens træningsfolds. Hvis én fælles scaler fittes på alle 25 optagelser før cross-validation, har valideringsdata påvirket preprocessing, og resultatet indeholder data leakage.

Cross-validation reducerer afhængigheden af ét bestemt split, men den skaber hverken nye brugere, sessioner eller uafhængige målinger. Den måler derfor stabiliteten inden for det eksisterende lille datasæt, ikke brugeruafhængig generalisering.

Mundtligt anker: "Ved stratificeret 5-fold cross-validation træner jeg fem modeller. Hver gang trænes der på 20 optagelser og valideres på fem, med én optagelse fra hver klasse. Hver optagelse er valideringsdata én gang. Det giver flere vurderinger end et enkelt split, men løser ikke begrænsningen ved kun én bruger og én session."

Seed 42 og `random_state` – Slides 13–14: Træning og evaluering

Placering i forklaringen: Forklar seed umiddelbart efter det stratificerede 80/20-split og inden gennemgangen af 5-fold-resultaterne. Seed handler om, hvordan data blandes og fordeles, og hører derfor til evaluering — ikke til sensoropsamling, features eller selve inferensen.

Et seed er startværdien for en pseudo-random generator. Computeren skaber en tilsyneladende tilfældig rækkefølge ud fra startværdien. Når samme data, kode og seed bruges igen, fås den samme blanding og det samme datasplit.

`random_state=42` bruges til at gøre det stratificerede holdout-split og blandingen før cross-validation reproducerbar. Tallet 42 har ingen særlig matematisk fordel; et andet heltal kunne også anvendes. Det forbedrer ikke accuracy og gør ikke datasættet mere tilfældigt.

- Samme seed gør eksperimentet reproducerbart.

- Et andet seed kan give et andet split og dermed et andet resultat.
- Effekten kan være stor med kun 25 optagelser.
- Cross-validation reducerer afhængigheden af ét bestemt holdout-split, men fjerner ikke datasættets begrænsninger.
- 42 er et almindeligt kulturelt programmeringsvalg, blandt andet som reference til *The Hitchhiker's Guide to the Galaxy*.

Mundtligt anker: "Seed 42 fastlåser den pseudo-tilfældige blanding, så eksperimentet kan gentages med samme split. Tallet er vilkårligt og forbedrer ikke modellen. Med så få optagelser kan andre seeds give andre resultater, og derfor supplerer jeg holdout med cross-validation."

Hvorfor er 80 % usikkert?

Testsettet har kun fem eksempler. Én fejl ændrer accuracy med 20 procentpoint. Resultatet viser, at prototypen kan fungere, men er ikke præcis evidens for fremtidig performance.

Kodeoverblik – fra `config.yaml` til metrics

`config.yaml`: Parametre, ikke programlogik

YAML-filen samler de værdier, som Python-pipelinen skal bruge: datamappe, klasser, 400 Hz, fire sekunder, 0,25 sekunders stride, de syv features, MLP-lagene `[32, 16]`, testandel, seed og beslutningstærskler.

- **Hvad:** En menneskeligt læsbar konfigurationsfil.
- **Hvordan:** `load_config()` bruger `yaml.safe_load()` og returnerer værdierne som Python-dictionaries.
- **Hvorfor:** Parametre kan ændres og dokumenteres ét sted frem for at blive spredt som hardcodede værdier.
- **Begrænsning:** En værdi i config har kun effekt, hvis koden faktisk læser og anvender den. Config-filen udfører ikke noget i sig selv.

`load_raw_data()`: Fra mange CSV-filer til én tabel

```
files = sorted(raw_dir.rglob(file_glob))
for fp in files:
    df = pd.read_csv(fp)
    df["__source_file"] = fp.name
    chunks.append(df)
return pd.concat(chunks, ignore_index=True)
```

- `rglob()` finder CSV-filer rekursivt i datamappen.
- Hver fil indlæses som en pandas `DataFrame`.
- `__source_file` gemmer, hvilken fysisk optagelse hver række kommer fra.
- Tabellerne samles, men filgrænsen bevares gennem kolonnen.

Hvorfor er filgrænsen vigtig? Et træningsvindue må ikke begynde i slutningen af én optagelse og fortsætte ind i en anden gestus. `build_dataset()` grupperer derfor efter `__source_file`, før der dannes vinduer.

`build_dataset()`: Ét eksempel per endelig optagelse

Det endelige datasæt har 1.600 samples per CSV, og vinduet kræver også 1.600 samples. Derfor giver hver fil præcis ét træningseksempel. Stride på 100 samples får først betydning, hvis en fil er længere end ét vindue. I LIVE-firmwaren betyder stride derimod, at 1.500 gamle samples beholdes, og 100 nye giver næste inference efter 0,25 sekund.

```
for _, recording in df.groupby("__source_file"):
    for window in sliding_windows(recording, 400, 4.0, 0.25):
        X_list.append(extract_features_for_window(window, use_mag=True))
        y_list.append(majority_label(window))
```

Output: `X` er en matrix med én række per optagelse og 28 kolonner per række. `y` er den tilhørende liste af facitlabels.

`LabelEncoder`: Tekstlabels bliver til heltal

`LabelEncoder.fit_transform(y)` omdanner eksempelvis klassestrengene til tal, som scikit-learn kan arbejde med. Scikit-learn sorterer normalt labels alfabetisk. Projektets ønskede firmwareorden er en anden, så `export_model.py` remapper outputlaget til rækkefølgen fra `config.yaml`.

Det er vigtigt, fordi outputneuron nummer 1 ellers kunne blive fortolket som den forkerte klasse på Photon 2, selv om de matematiske scores var korrekte.

Split og stratificering

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y_enc,
    test_size=0.2,
    random_state=42,
    stratify=y_enc,
)
```

- `test_size=0.2`: fem af 25 optagelser bliver testdata.
- `stratify=y_enc`: én optagelse fra hver klasse kommer i testdelen.
- `random_state=42`: det samme pseudo-tilfældige split kan genskabes.
- Funktionen returnerer både features og labels for train og test.

StandardScaler uden data leakage

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

`fit_transform(X_train)` lærer middelværdi og standardafvigelse fra træningsdata og skalerer dem. `transform(X_test)` genbruger de allerede lærte værdier uden at lære fra testdata. Hvis der blev kaldt `fit_transform()` på testdata, ville evalueringen ikke længere simulere helt ukendte data.

`model.fit()`: Her foregår træningen

`train_model()` opretter et `MLPClassifier` med lagene `28 → 32 → 16 → 5`. Kaldet:

```
model.fit(X_train_s, y_train)
```

udfører den iterative træning. Scikit-learn håndterer forward-pass, loss, backpropagation og opdatering af vægte. At der kun står én linje betyder derfor ikke, at træningen matematisk kun er én operation; biblioteket skjuler træningsløkken bag API-kaldet.

Batch size i den konkrete kørsel: `config.yaml` angiver batch size 32, men holdout-træningsdelen indeholder kun 20 eksempler. Scikit-learn udsender derfor en advarsel og reducerer automatisk batch size til 20. Træningen er i praksis full-batch på dette meget lille træningssæt. Det stopper ikke træningen, men viser endnu en konsekvens af det lille datasæt.

`predict()` og metrics

```
y_pred = model.predict(X_test_s)
acc = accuracy_score(y_test, y_pred)
```

`predict()` udfører inference på de fem testeksempler. De forudsagte labels sammenlignes derefter med facit i `y_test`. Precision, recall, F1 og confusion matrix beregnes fra de samme to lister.

`train.py` kontra `export_model.py`

- `train.py` bygger datasættet, træner en model og udskriver holdout-metrics. Den gemmer ikke den deployerede C++-model.
- `export_model.py` genbruger data- og featurefunktionerne, træner MLP'et og eksporterer scaler, vægte, biases og klasserækkefølge til `model_data.cpp` samt artefakter.
- Femfolds-cross-validation er en supplerende evaluering; den køres ikke af den nuværende `train.py`-hovedfunktion.

Mundtligt anker: "Konfigurationen styrer parametrene. Python indlæser hver CSV og bevarer optagelsesgrænserne, komprimerer hvert vindue til 28 features og laver et stratificeret 80/20-split. StandardScaler fittes kun på træningsdelen. `model.fit()` udfører selve træningen med backpropagation,

og `predict()` evaluerer på holdout-data. Eksportscriptet omsætter derefter den lærte scaler og MLP-parametre til C++."

12. Python-model til C++

Træning i Python

Python-koden indlæser CSV-data, skaber vinduer, beregner features, splitter datasættet, fitter StandardScaler, træner MLP'et og beregner metrics.

Eksport

Scalerens værdier, klassernes rækkefølge, modellens vægte og biases eksporteres som konstante C++-arrays.

Inference i C++

Firmwaren beregner de samme features og udfører:

skalering → lag 1 + ReLU → lag 2 + ReLU → outputlag + softmax

Det er en rigtig forward-pass på enheden. Pickle-filen kører ikke direkte på Photon 2.

Kodekort: Scaler og forward-pass – Slides 15–16

Det centrale princip i `model_infer()` i `Product/firmware/src/model_data.cpp` er:

```
act[i] = (features[i] - scalerMean[i]) / scalerScale[i];

acc = kBiasesFlat[bOff + o];
for (size_t i = 0; i < inDim; ++i)
    acc += act[i] * kWeightsFlat[wOff + i*outDim + o];
next[o] = acc;

relu_inplace(act, outDim); // kun skjulte lag
// Til sidst: softmax til fem output-sandsynligheder
```

Udsnittet er pædagogisk forkortet; de eksporterede arrays hedder blandt andet `kScalerMean`, `kScalerScale`, `kWeightsFlat` og `kBiasesFlat` i den faktiske kode.

Hvad gør koden? Den omdanner de 28 features til fem klassescores og derefter fem tal mellem 0 og 1, som summerer til 1.

Hvordan gør den? Først anvendes den StandardScaler, der blev fitted på træningsdata. Derefter beregner hvert neuron en vægtet sum plus bias. ReLU fjerner negative aktiveringer i de skjulte lag, og softmax normaliserer outputlaget.

Hvorfor ligger dette på Photon 2? Det er inference: anvendelse af allerede lærte og eksporterede parametre på nye sensordata. Det kræver kun et forward-pass. Backpropagation, gradienter og ændring af vægte foregår kun under offline træning på computeren.

Mundtligt anker: "På Photon 2 starter modelkoden med at standardisere de 28 features med træningssættets gemte middelværdier og skalaer. Derefter beregner hvert lag vægtet sum plus bias. De skjulte lag bruger ReLU, og outputlaget bruger softmax. Det er kun et forward-pass med faste parametre; der trænes ikke på enheden."

Vigtig risiko

Featureorden og preprocessing skal være identiske i Python og C++. Selv en korrekt model giver forkerte resultater, hvis inputkontrakten afviger.

Projektet har ikke en færdig automatiseret "golden vector"-test, som sender præcis det samme vindue gennem både Python- og C++-implementationen og sammenligner alle 28 features og fem scores numerisk. Formlerne og rækkefølgen er implementeret parallelt og dokumenteret, men en sådan parity-test ville være den stærkeste næste verifikation af eksportkæden.

TinyML-toolchain

En toolchain er kæden af værktøjer fra dataindsamling til en kørende embedded model. Der findes ikke én universel TinyML-proces; valg af værktøjer afhænger af model, hardware og krav.

- Data indsamles og analyseres typisk i Python eller notebooks.
- Modellen trænes og evalueres på en computer.
- Modellen konverteres eller eksporteres til et format, embedded software kan bruge.
- Firmware kompiles og flashes til enheden.
- Den deployede løsning måles på accuracy, latency, hukommelse, energi og robusthed.
- Resultaterne bruges til en ny iteration af data, features, model eller firmware.

I mit projekt: Python og scikit-learn bruges til træning. Egne eksportværktøjer genererer C++-arrays og forward-pass-kode til Particle Workbench-projektet.

emlearn

emlearn er et værktøj til at konvertere udvalgte klassiske machine-learning-modeller til effektiv C-kode, så de kan køre på microcontrollere.

- Det kan blandt andet bruges til træbaserede modeller.
- Modellen trænes normalt i Python og konverteres derefter.
- Fordelen er, at man ikke behøver implementere hele inferencealgoritmen manuelt.
- Kun understøttede modeltyper og operationer kan konverteres.

I mit projekt: emlearn blev ikke brugt i den endelige løsning; MLP-parametrene blev eksporteret til en egen C++ forward-pass.

AlfES

AlfES er et embedded neuralt-netværksbibliotek, der kan udføre inference og i nogle tilfælde træning direkte på microcontrollere.

- Det er rettet mod små neurale netværk på begrænset hardware.
- Det kan understøtte on-device training, hvor modellen lærer på selve enheden.
- Biblioteker reducerer implementeringsarbejde, men tilføjer deres egne API'er og begrænsninger.

I mit projekt: AlfES blev ikke brugt; netværkets forward-pass blev implementeret direkte i C++.

Edge Impulse

Edge Impulse er en end-to-end-plattform til indsamling, signalbehandling, modeltræning, evaluering og generering af embedded biblioteker.

- Platformen kan gøre prototyping og deployment hurtigere.
- Den indeholder færdige processing blocks og modelværktøjer.
- Abstraktionen gør processen lettere, men kan skjule nogle implementeringsdetaljer.
- Man skal stadig forstå data, validering og hardwarebegrænsninger.

I mit projekt: Popen blev bygget med egne Python- og C++-komponenter frem for Edge Impulse.

CMSIS og hardwareoptimering

CMSIS er Arm-standarder og biblioteker til Cortex-processorer. CMSIS-DSP og CMSIS-NN indeholder optimerede signalbehandlings- og neurale netværksoperationer, som kan udnytte processorens instruktioner bedre end generisk kode.

- Det kan reducere inferenstid og nogle gange hukommelsesforbrug.
- Det er mest relevant, når profiling viser en reel flaskehals.
- Optimering kan gøre koden mindre portabel og mere kompleks.
- Før og efter optimering skal numerisk korrekthed kontrolleres.

I mit projekt: Der blev ikke anvendt CMSIS-optimering. Forward-pass på cirka 0,35 ms var allerede langt under projektets mål på 50 ms.

Offline training og on-device training

Ved offline training trænes modellen på en computer og deployes derefter som faste parametre. Ved on-device training opdaterer den embedded enhed selv hele eller dele af modellen.

- Offline training giver adgang til mere regnekraft og nemmere analyse.

- On-device training kan muliggøre personlig tilpasning og løbende adaptation.
- On-device training kræver ekstra RAM, beregning, energi og sikker håndtering af nye data.
- Inference er normalt langt billigere end træning.

I mit projekt: Træningen er offline i Python. Photon 2 udfører kun inference og ændrer ikke modellens vægte.

Designbegrundelse: Hvorfor ikke selvtræning på Photon 2?

Valget er en bevidst opdeling mellem en udviklingsplatform og en deployment-plattform:

- Computeren har de nødvendige Python-værktøjer, mere regnekraft og bedre muligheder for at inspicere loss, metrics og fejl.
- Backpropagation kræver gemte aktiveringer, gradienter for vægte og biases samt ekstra optimer-tilstand. Det kræver mere RAM, beregning og energi end et forward-pass.
- Supervised selvtræning kræver korrekte labels til nye input. Under normal LIVE-brug ved Photon 2 ikke automatisk, om brugerens bevægelse var `tap1`, `tap2` eller en anden klasse.
- Offline træning gør datasplit, preprocessing, modelversion og evaluering lettere at reproducere og kontrollere.
- På Photon 2 er opgaven at anvende en kontrolleret, færdigtrænet model hurtigt og stabilt, ikke at ændre modellen under brug.

Det er ikke en påstand om, at enhver form for on-device training er fysisk umulig. Den lille model kunne muligvis trænes eller finjusteres med en specialiseret implementation, men det ville kræve en anden data-, label- og valideringsarkitektur og lå uden for projektets scope.

Mundtligt anker: "Jeg har bevidst adskilt træning og deployment. Computeren udfører den dyre og kontrollerbare træningsproces med backpropagation, mens Photon 2 kun udfører den billigere forward-pass med faste parametre. Det reducerer RAM-, beregnings- og energibehovet, undgår problemet med manglende labels under LIVE-brug og gør modellen reproducerbar. On-device training er ikke principielt umuligt, men det er ikke nødvendigt eller implementeret i dette projekt."

13. Firmware og beslutningslogik

Vinduer og sliding windows

Det første vindue kræver 1.600 samples. Derefter beholdes de nyeste 1.500, og 100 nye samples giver en ny inference hvert 0,25 sekund.

Confidence threshold

En prediction under 0,75 accepteres ikke som en handling. Det reducerer usikre events, men kan også afvise korrekte gestusser.

Beslutningsstabilisering

Samme sikre klasse skal forekomme tre gange i træk. Det stabiliserer output, men tilføjer forsinkelse.

Kodekort: `updateDecision()` – Slide 16

```
const bool confident = score >= 0.75f;
const int candidate = confident ? classIdx : CLASS_IDLE;

// historikken opdateres med candidate
if (historyCount < 3) return -1;
if (history[0] != history[1] || history[1] != history[2])
    return -1;
if (nowMs - lastEventMs < 4000) return -1;

return history[0];
```

Udsnittet er forkortet, men følger den faktiske kontrolrækkefølge.

- **Hvad betyder `return -1`?** Der er endnu ingen godkendt handling. Det er en intern sentinel-værdi og ikke en modelklasse.
- **Hvad betyder `?: ?`?** Det er en ternary operator: hvis `confident` er sand, bruges `classIdx`; ellers bruges `CLASS_IDLE`.
- **Hvorfor tre ens?** En enkelt ustabil prediction må ikke udløse en kommando.
- **Hvorfor debounce?** Det samme fysiske input bliver liggende i flere overlappende vinduer og må kun udløse én handling.
- **Tradeoff:** Filtrene reducerer falske events, men øger latenstid og kan undertrykke korrekte, men kortvarigt ustabile gestusser.

Tap-kontrol

En supplerende algoritme tæller tydelige impacts og hjælper med at skelne tap1, tap2 og tap3. Løsningen er derfor hybrid: ML klassificerer mønstret, og regler kontrollerer handlingen.

Vigtig resultatgrænse: Python-metrics måler MLP-klassifikationen på holdout-data. LIVE-firmwaren tilføjer bagefter impact count og beslutningsfiltre. Den komplette deployede adfærd er derfor en hybrid pipeline, og dens samlede event-performance er ikke identisk med holdout-accuracy for MLP'et alene.

Debounce

Efter en godkendt gestus forhindrer fire sekunders debounce den samme fysiske gestus i at udløse flere kommandoer.

Non-blocking firmware

LED-mønstrene styres som en tilstandsmaskine uden lange `delay()`-kald. Sampling og anden logik kan derfor fortsætte, mens LED'en blinker.

Driftsformer

- **DEBUG:** Diagnostik og verbose output uden normale gestushandlinger.
- **TRAINING:** Guidet dataindsamling.
- **LIVE:** Lokal inference, events og RGB-feedback.

14. Begrænsninger og næste eksperiment

Begrænsninger

- Kun 25 accepterede optagelser.
- Kun én person.
- Kun én afsluttende session i deploy-datasættet.
- Kun ét testeksempel per klasse.
- Mulig session leakage eller korrelation mellem train og test.
- `tap1` og `tap2` kan ligne hinanden.
- Kvalitetsfiltrering kan gøre data mindre realistiske.
- Fysisk LIVE-verifikation var ikke en blind accuracy-test.
- Lav inferenstid er ikke det samme som lav samlet brugeroplevelt responstid.

Bedste næste eksperiment

Indsaml data fra flere personer og sessioner. Hold en hel ny person eller session fuldstændigt ude under udviklingen og brug den som blind test. Mål derefter per-klasse performance, false triggers under længere idle-perioder og samlet responstid.

Stærk konklusion

“Projektet demonstrerer en fungerende end-to-end TinyML-prototype. Det dokumenterer ikke endnu robust generalisering til nye brugere eller situationer.”

Iterativ ML-udvikling og MLOps-tankegang

Et ML-system udvikles normalt iterativt. Fejl kan skyldes data, labels, preprocessing, modellen, deployment eller ændringer i den virkelige verden. Derfor gentager man måling, analyse, ændring, deployment og overvågning.

- En bedre model kan ikke kompensere for dårlige eller irrelevante data.
- Offline metrics og live performance skal begge undersøges.
- Model-, scaler-, feature- og firmwareversioner skal passe sammen.

- Nye data kan vise, at krav eller klasser skal omdefinieres.
- Monitoring kan afsløre data drift, timingproblemer og false triggers efter deployment.

I mit projekt: Forløbet gik fra 50 Hz-data til 400 Hz, kvalitetskontrolleret dataindsamling, ny træning, eksport, firmwarebuild og fysisk LIVE-verifikation.

15. Grundbegreber fra pensum

TinyML

Machine learning på ressourcebegrænset embedded hardware tæt på sensoren. Fordele kan være offline funktion, lav kommunikationslatens, mindre datatransmission og lokal behandling. Ulemper er begrænset RAM, flash, regnekraft og energi.

Microcontroller kontra almindelig computer

En microcontroller samler processor, hukommelse og hardwareperiferi på en chip og er lavet til styring tæt på sensorer og aktuatorer. Den har normalt færre ressourcer end en pc og kører ofte uden et fuldt desktop-operativsystem.

- RAM og flash er begrænsede og har forskellige roller.
- Real-time-adfærd og forudsigelig timing kan være vigtigere end maksimal throughput.
- Sensorinterfaces som I2C, SPI og ADC er centrale.
- Dynamisk hukommelse og store runtime-frameworks undgås ofte.
- Modellen skal vurderes som del af hele firmwareproduktet, ikke isoleret.

I mit projekt: Photon 2 samler ADXL343, holder en tidsseriebuffer, beregner features, kører modellen og styrer serial-output og LED.

Klassifikation, regression og anomaly detection

Forudsigelse af en diskret kategori. Bruges i projektet.

Regression

Forudsigelse af en kontinuerlig numerisk værdi, eksempelvis temperatur eller resterende levetid.

Anomaly detection

Identifikation af observationer, som afviger fra det normale. Det kan bruges, når man har gode normaldata, men få eksempler på alle mulige fejl.

Inference

Anvendelse af en allerede trænet model på nye input. Træning lærer parametrene; inference bruger dem.

Den samlede inferenskæde i projektet er:

1. Photon 2 opsamler 1.600 XYZ-samples i et firesekunders vindue.
2. Magnitude beregnes, og kanalerne mean-centres.
3. De 28 features beregnes i samme orden som under træning.
4. Features standardiseres med de eksporterede scaler-parametre.
5. MLP'et udfører forward-pass og softmax.
6. Den største score giver en klassekandidat.
7. Confidence-threshold, tre ens predictions, tap-kontrol og debounce afgør, om kandidaten må blive til en handling.

Det første komplette vindue kræver fire sekunders data. Derefter beholdes de nyeste 1.500 samples, og 100 nye samples giver en ny modelkørsel hvert $100 / 400 = 0,25$ sekund.

Den registrerede modelinferens tog cirka 345 μ s i gennemsnit og 364 μ s maksimalt. Timeren ligger omkring `model_infer()` efter `extractFeatures()`. Tallet omfatter derfor scaler og MLP-forward-pass, men ikke fire sekunders dataopsamling eller feature extraction. Lav modelinferens er således ikke det samme som 345 μ s samlet brugeroplevelst responstid.

Softmax-outputtet er en relativ modelscore og ikke en garanti for en korrekt eller perfekt kalibreret sandsynlighed. Derfor kræver firmwaren mindst 0,75 confidence og stabilitet over tre predictions, før en ikke-idle-klasse accepteres.

Mundtligt anker: "Inferens er anvendelsen af den allerede trænede model på nye data. Photon 2 beregner de samme features og den samme skalering som Python og udfører derefter et forward-pass. Vægtene ændres ikke. Modellens kandidat filtreres bagefter af confidence og stabilitetsregler, før en kommando udsendes."

Latency og responstid

Tiden fra et relevant startpunkt til et resultat. Man skal angive, om man mener modelberegning, vinduesopsamling eller samlet brugeroplevelst responstid.

Flash og RAM

- **Flash:** Permanent lager til firmware og modelkonstanter.
- **RAM:** Arbejdshukommelse til blandt andet buffers, mellemresultater og runtime-tilstand.

Kvantisering

Repræsentation af vægte og eventuelt aktiveringer med lavere præcision, eksempelvis int8 frem for float32. Det kan reducere størrelse og beregningsbehov, men kan påvirke accuracy. Projektets endelige model bruger float32 og er ikke kvantiseret.

- Float32 bruger normalt 32 bit per tal, mens int8 bruger 8 bit.
- Int8-vægte kan derfor fylde omtrent en fjerdedel før anden overhead.
- Skalering og zero-point bruges ofte til at mappe mellem reelle værdier og heltal.
- Post-training quantization udføres efter træning.

- Quantization-aware training simulerer kvantisering under træningen.
- Accuracy og numerisk paritet skal måles efter konverteringen.

I mit projekt: Kvantisering blev ikke nødvendig for at få modellen til at passe eller opfylde latensmålet. Det er perspektivering, ikke et udført resultat.

Pruning

Pruning fjerner vægte, neuroner eller forbindelser, som vurderes at bidrage lidt til modellens output. Målet er en mindre eller billigere model.

- Unstructured pruning sætter individuelle vægte til nul.
- Structured pruning fjerner hele neuroner, kanaler eller filtre.
- En sparse model bliver kun hurtigere, hvis runtime og hardware kan udnytte sparsiteten.
- Fine-tuning efter pruning kan genvinde tabt accuracy.
- Pruning skal evalueres på både modelkvalitet og den reelle embedded implementering.

I mit projekt: Modellen blev ikke pruned, fordi den allerede var lille og hurtig nok.

Low-power-design

Low-power-design handler om at reducere enhedens energiforbrug, ikke kun modellens inferenstid. Sensor, CPU, radio, LED og vågentid bidrager alle til forbruget.

- Enheden kan sove mellem relevante målinger.
- Lavere samplingfrekvens reducerer ofte sensor- og CPU-arbejde.
- Eventbaseret aktivering kan undgå kontinuerlig tung inference.
- Lokal behandling kan være billigere end at sende rå data trådløst.
- Energi per inference og gennemsnitligt effektforbrug er forskellige målinger.
- Optimering kræver faktiske strømmålinger; hurtig kode er ikke automatisk lavenergi.

I mit projekt: Energiforbruget blev ikke målt. Systemet prioriterede funktionalitet og kontinuerlig sampling ved 400 Hz.

K-means anomaly detection

K-means grupperer data omkring et valgt antal centre. Ved anomaly detection kan afstanden til det nærmeste center bruges som anomaly score: stor afstand kan betyde, at observationen ikke ligner normaldataene.

- K-means er unsupervised.
- Antallet af klynger, k , vælges på forhånd.
- Features bør ofte skaleres før afstandsberegning.

- Metoden antager nogenlunde kompakte grupper og kan være følsom over for outliers.
- En threshold afgør, hvornår en afstand betragtes som anomal.

I mit projekt: K-means anvendes ikke; alle fem ønskede klasser er labellede og trænes supervised.

Gaussian Mixture Model – GMM

En GMM beskriver data som en blanding af flere Gaussiske fordelinger. Den kan give en likelihood for, hvor sandsynligt et nyt eksempel er under den lærte normalfordeling. Lav likelihood kan bruges som tegn på en anomaly.

- GMM er probabilistisk og kan beskrive overlappende grupper.
- Hver komponent har blandt andet middelværdi og kovarians.
- Antal komponenter vælges som en modelbeslutning.
- Metoden afhænger af, hvor rimeligt de Gaussiske antagelser passer til dataene.

I mit projekt: GMM anvendes ikke, fordi opgaven er direkte femklasseklassifikation.

Autoencoder til anomaly detection

En autoencoder er et neuralt netværk, der trænes til at genskabe sit input gennem en komprimeret intern repræsentation. Hvis den primært trænes på normaldata, vil en stor reconstruction error på et nyt input kunne indikere en anomaly.

- Encoderen komprimerer inputtet.
- Decoderen forsøger at rekonstruere det.
- Reconstruction error måler forskellen mellem input og rekonstruktion.
- En threshold omsætter fejlen til normal/anomal.
- Hvis autoencoderen er for kraftig, kan den også blive god til at rekonstruere anomalier.

I mit projekt: En autoencoder anvendes ikke. Projektet har labels for alle fem kendte klasser og bruger derfor supervised klassifikation.

16. Kodekort og dokumenterede henvisninger

Dette afsnit er indgangen til hele den nuværende kodebase. Linjenumre er omtrentlige, fordi kommentarer kan flytte dem; funktionsnavnene er de stabile henvisninger. Den vigtigste sammenhæng er:

`randomized_capture_gui.py` → CSV/JSON → `train.py` eller `export_model.py` → `model_data.h/.cpp` → `main.cpp` → Photon 2-output

Hvad er autoritativt, genereret og legacy?

- **Autoritativ konfiguration:** `Product/ml/config.yaml` beskriver den aktuelle model- og datakontrakt.
- **Aktiv træningsreference:** `Product/ml/train.py` er den letteste fil at læse for preprocessing, features, split, scaler, træning og metrics.
- **Aktiv deploymentpipeline:** `Product/ml/export_model.py` gentager træningsforløbet og eksporterer den trænedede model til C++.
- **Genereret modelkode:** `Product/firmware/src/model_data.h` og `model_data.cpp` er output fra eksporten. De skal regenereres, ikke håndredigeres, når modellen ændres.
- **Aktiv runtime:** `Product/firmware/src/main.cpp` samler sensoren, beregner features, kalder modellen og filtrerer beslutningen.
- **Legacy:** `Product/ml/tools/analyze-taps-legacy.py` er et tidligt analyseværktøj og er ikke en del af den endelige 28-feature-pipeline.
- **Præsentationsværktøjer:** Kodet i `Presentation/` bygger visuelle materialer, men er ikke en del af TinyML-produktets runtime.

`Product/ml/config.yaml` – den fælles kontrakt

Hvad: Samler stier, klasser, sampling, preprocessing, features, modelarkitektur, evaluering, beslutningslogik og eksportvalg.

Vigtige aktuelle værdier: `400 Hz`, `4,0 s`, `0,25 s` stride, syv features på fire kanaler, skjulte lag `[32, 16]`, testandel `0,2`, seed `42`, confidence `0,75`, tre stabile vinduer og `4.000 ms` debounce.

Hvorfor den er vigtig: Hvis Python og C++ bruger forskellige værdier eller featureorden, er deployment ikke den samme modelpipeline som den evaluerede pipeline.

`Product/ml/randomized_capture_gui.py` – labellet dataindsamling

Hvad: Styrer randomiserede trials, sender `TAP_SCOPE` til Photon 2, modtager X/Y/Z over seriel, beregner kvalitetsmål og gemmer accepterede eller afviste CSV/JSON-filer.

Nøglesteder:

- `Trial` omkring linje 28 beskriver label, tempo og kraft.
- `parse_args()` omkring linje 66 læser serielport og forsøgsparametre.
- `RandomizedCapture` omkring linje 84 indeholder GUI- og captureforløbet.
- `make_frame()` omkring linje 293 beregner baselinekorrigerede akser og dynamisk magnitude.
- `metrics()` omkring linje 305 beregner peak, RMS, aktiv varighed, event count og clipping.
- `validate()` omkring linje 339 anvender de klassespecifikke kvalitetsgates.
- `save()` omkring linje 364 gemmer signal og metadata.

Eksamenssætning: "Capture-programmet hjælper mig med at holde label, udførelse og kvalitetskontrol sammen; det er ikke modellen, men værktøjet der producerer dokumenterede træningskandidater."

Product/ml/train.py – læsbar reference for ML-pipelinen

Hvad: Indlæser de accepterede CSV-filer, bygger featurematricen, laver stratificeret holdout, fitter scaler og MLP og skriver metrics og artefakter.

Nøglefunktioner:

- `load_config()` linje 37: læser YAML sikkert.
- `load_raw_data()` linje 42: samler og validerer CSV-kolonner.
- `sliding_windows()` linje 70: opdeler længere optagelser efter vindue og stride.
- `peak_count()` linje 91: tæller peaks med threshold og refractory distance.
- `channel_features()` linje 111: beregner de syv features.
- `extract_features_for_window()` linje 122: mean-centrerer X/Y/Z, tilføjer magnitude og skaber 28 tal.
- `build_dataset()` linje 140: laver `x` og `y`; ved endelige firesekunders filer bliver det ét eksempel per optagelse.
- `validate_class_coverage()` linje 165: kræver nok data i alle konfigurerede klasser.
- `train_model()` linje 180: konstruerer `MLPClassifier`.
- `main()` linje 203: udfører split → scaler → fit → predict → metrics → artefakter.

Vigtig afgrænsning: Den supplerende 5-fold-CV på 76 % findes ikke som et fast trin i denne fil. Den blev kørt som en separat kontrolberegning. Det reproducerbare hovedresultat fra `train.py` er 80/20-holdout-resultatet.

Product/ml/export_model.py – fra Python til C++

Hvad: Træner med samme kontrakt og skriver scaler, labels, lagstørrelser, vægte og biases som kompilerbar C++.

Nøglefunktioner:

- `compute_feature_count()` linje 50 kontrollerer inputdimensionen.
- `generate_header()` linje 79 skriver C++-interfacet.
- `_layer_meta()` linje 135 beskriver lagenes dimensioner.
- `generate_cpp()` linje 149 skriver scaler, parametre og forward-pass.
- `_train_and_export()` linje 303 udfører split, scaling og træning før eksport.
- `main()` linje 389 skriver filerne til firmwareappen.

Vigtig detalje: `LabelEncoder` bruger alfabetisk intern klasseorden under træning. Eksporten remapper output til `config.yaml`-ordenen, så firmware, navne og scores fortsat passer sammen.

Product/ml/generate_figures.py – rapportens figurer

Hvad: Læser projektdata og genererer signalfigurer samt confusion matrix fra den dokumenterede pipeline.

Nøglefunktioner: `load_trials()` linje 34, `plot_gesture_signals()` linje 51, `evaluate_confusion()` linje 84 og `plot_confusion_matrix()` linje 107.

Afgrænsning: Filen visualiserer resultater; den ændrer ikke den deployede model.

Product/firmware/src/model_data.h og model_data.cpp – den eksporterede model

Hvad: Headeren definerer modelmetadata og funktioner. C++-filen indeholder scalerens 28 middel-/scaleværdier, MLP'ens 1.541 parametre, klassenavne og forward-pass.

Nøglesteder: `model_init()` omkring linje 255 og `model_infer()` omkring linje 259 i `model_data.cpp`.

Hvordan: `model_infer()` standardiserer de 28 inputfeatures, beregner lagene `28 → 32 → 16 → 5` med ReLU i de skjulte lag og softmax på outputtet.

Hvorfor genereret kode: Modellen kan afvikles lokalt uden Python eller scikit-learn på Photon 2. De lange arrays er tal, som træningen har lært; de skal ikke forklares én for én.

Product/firmware/src/main.cpp – sensor, features og LIVE-logik

Hvad: Er produktets aktive firmware og binder ADXL343, dataopsamling, feature extraction, model-inference, beslutningsfilter, serielkommandoer og RGB-feedback sammen.

Nøglefunktioner:

- `channelFeatures()` linje 287: de syv statistikker per kanal.
- `extractFeatures()` linje 347: X/Y/Z/magnitude → 28 features i korrekt orden.
- `countImpactEvents()` linje 374: fysisk sanity check for tap-klasser.
- `updateDecision()` linje 414: confidence, stabilitet, tap-kontrol og debounce.
- `runInference()` linje 445: feature extraction, `model_infer()`, argmax og beslutning.
- `readAdxlRawXYZ()` linje 546: samler seks registerbytes til tre signed 16-bit-tal.
- `readSampleG()` linje 556: rå værdier → g med `0,0039 g/LSB`.
- `startTapScope()` / `runTapScope()` linje 574/595: bufferet X/Y/Z-capture ved 400 Hz.
- `startBaseline()` / `runBaseline()` linje 682/705: den ældre guidede baseline/capture-rutine ved 50 Hz.
- `setOperatingMode()` linje 914: skifter mellem DEBUG, TRAINING og LIVE.
- `handleSerialCommand()` linje 949: håndterer blandt andet `MODE LIVE` og `TAP_SCOPE`.
- `setup()` / `loop()` linje 1074/1091: initialisering og non-blocking hovedloop.

Vigtig præcision: `runInference()` måler kun tiden omkring selve `model_infer()` som model-inference-latency. Feature extraction udføres lige før og er derfor ikke med i dette timingtal.

Product/firmware/project.properties – Particle-build

Hvad: Angiver firmwareprojektets navn og afhængigheder til Particle build-systemet.

Hvorfor: Filen gør det muligt for Particle CLI at kompilere mappen som et firmwareprojekt. Selve ADXL343-adgangen i `main.cpp` bruger direkte I2C-registerlæsning.

Start-Photon2-Demo.ps1 og START-PHOTON2-DEMO.cmd – én-kliks demo

Hvad: Validerer værktøjer og model, cloud-kompilerer, flasher Photon 2 lokalt og sender `MODE LIVE` over den fundne serielport.

Nøglesteder i PowerShell: parameterblokken linje 2, `Invoke-Particle()` linje 27, portdetektion linje 35–41 og `Set-PhotonLiveMode()` linje 56.

Hvordan: CMD-filen er kun en dobbeltklikbar indgang, der starter PowerShell-scriptet med korrekte argumenter.

Hvorfor: Det reducerer risikoen for at starte demoen med forkert firmware eller driftsform. Det er deploymentautomatisering, ikke en del af klassifikationsalgoritmen.

Product/ml/tools/analyze-taps-legacy.py – tidligere analyse

Hvad: Et lille tidligt script til at læse CSV og detektere impulser med simple tærskler.

Hvorfor det stadig findes: Det dokumenterer den iterative undersøgelse af taps, men er markeret `legacy`, fordi det ikke definerer den endelige preprocessing, de 28 features eller LIVE-modellen.

Kode til præsentation og PDF

Hvad: `Presentation/fill_existing_deck.py` fylder PowerPoint-skabelonen med tekst, figurer og kodekort. `Presentation/generate_axis_visual.py` laver før/efter-illustrationen af den antagede og målte akseretning. `Presentation/render_exam_notes.py` konverterer dette Markdown-dokument til den tabletvenlige PDF via Chrome/Chromium.

Hvorfor de nævnes: De dokumenterer, hvordan præsentationsmaterialet er produceret, men de påvirker hverken data, modeltræning eller firmware-inference.

Støttefiler uden selvstændig programlogik

- `Product/ml/feature_spec.md` beskriver featurekontrakten i tekst.
- `Product/ml/requirements.txt` beskriver Python-afhængighederne til ML-værktøjerne.
- README-filerne beskriver brug og projektstruktur.
- `Product/ml/artifacts/` indeholder serialiserede resultater fra træning og eksport.
- `Product/firmware/firmware_photon2.bin` er den kompilerede firmware, mens `build/legacy/` bevarer ældre builds tydeligt adskilt fra den aktuelle demo.

Hvad er endnu ikke automatisk verificeret?

Der findes ikke en automatiseret "golden vector"-test, som sender præcis samme råvindue gennem Python og C++ og sammenligner alle 28 features og fem outputs numerisk. Pariteten er gennemgået i kode og afprøvet end-to-end, men en sådan test ville være det stærkeste næste skridt til at opdage forskelle i featureorden, floating-point eller preprocessing.

AI-brug

AI blev brugt til forslag, fejlsøgning, struktur og sproglig redigering. De fysiske valg, dataindsamlingen, målingerne og den endelige vurdering blev udført og kontrolleret gennem kodegennemgang, træning, build og fysisk test. Med mere tid ville jeg dokumentere flere teoretiske mellemregninger og valideringer selvstændigt.

Kildegrundlag i rapporten

Rapportens kilder har forskellige funktioner. En primær teknisk kilde dokumenterer konkrete krav eller specifikationer. En lærebog forklarer teori og metode. Projektets egne filer og målinger dokumenterer, hvad der faktisk blev implementeret og observeret.

- Brug producentens datablad til hardwareegenskaber.
- Brug bibliotekets officielle dokumentation til den præcise softwareadfærd.
- Brug lærebøger til generelle ML-principper og sammenhænge.
- Brug egne data, kode og målinger til projektets resultater.
- En ekstern kilde kan ikke erstatte evidens for, at ens egen prototype virker.

Reference [1] – kursets TinyML-introduktion

Kursussiden *Welcome to TinyML* beskriver kursets forståelse af TinyML som krydsfeltet mellem machine learning, signalbehandling og embedded IoT. Den fremhæver datasæt, preprocessing, feature engineering, embedded deployment, optimering og low-power-design.

- Kilden bruges til at placere projektet i kursets faglige ramme.
- Den er valgt, fordi den er den officielle beskrivelse af netop dette kursus.
- Den er ikke en detaljeret videnskabelig definition af alle TinyML-emner.
- En lærebog eller forskningsartikel kunne give mere teori, men kan ikke definere underviserens konkrete kursusmål.

I rapporten: [1] understøtter introduktionens beskrivelse af, hvad kurset arbejder med.

Reference [2] – semesterprojektets krav

Projektbriefen er den autoritative kilde til afleveringskrav, hardwarekrav og eksamensform.

- Den kræver Photon 2, lokal sensor, egne labellede data, preprocessing og en ML-/DL-algoritme.
- Den kræver observerbart output og deling af kode, data og dokumentation.
- Den siger, at rapporten bør sigte mod cirka 20 sider.
- Den angiver højst fem minutters demo og cirka 50/50 mellem rapport og øvrigt pensum.

- Ingen lærebog kan bruges som alternativ til de lokale eksamenskrav.

I rapporten: [2] bruges i introduktion, kravsporbarhed og forklaringen af, hvorfor løsningen ikke må være ren regelprogrammering.

Reference [3] – ADXL343-databladet

Analog Devices' datablad er den primære tekniske kilde til accelerometerets elektriske og målte egenskaber.

- Det dokumenterer måleområderne ± 2 , ± 4 , ± 8 og ± 16 g.
- Det dokumenterer full-resolution scale factor på cirka 3,9 mg per LSB.
- Det beskriver output data rates, registre, I2C-adresser, device-ID og støjegenskaber.
- Producentens datablad vælges frem for en tilfældig tutorial, fordi specifikationerne kommer fra den ansvarlige producent.
- En library-guide kan vise kodebrug, men er svagere som kilde til sensorens fysiske grænser.

I rapporten: [3] understøtter sensorområde, opløsning, maksimal samplingfrekvens og bemærkningen om støj over 100 Hz.

Reference [4] – Photon 2-dokumentation

Particles officielle Photon 2-datablad dokumenterer den konkrete embedded platforms processor, hukommelse og interfaces.

- Photon 2 har en 200 MHz Arm Cortex-M33.
- Dokumentationen angiver den tilgængelige flash og RAM.
- Producentdokumentationen er valgt, fordi hardwarevarianter og Device OS-forhold kan være produktspecifikke.
- En generel Cortex-M33-kilde beskriver CPU-arkitekturen, men ikke nødvendigvis Photon 2-kortets samlede ressourcer.

I rapporten: [4] bruges til at vurdere, om model, buffer og firmware kan være på enheden.

Reference [5] – scikit-learn StandardScaler

Den officielle API-dokumentation beskriver den konkrete `StandardScaler`, som koden anvender.

- Dokumentationen angiver transformationen $z = (x - \mu) / \sigma$.
- `fit` lærer middelværdi og skala fra træningsdata.
- `transform` genbruger de lærte værdier på nye data.
- API-dokumentationen er valgt frem for kun en lærebog, fordi den beskriver den faktiske klasse og dens implementeringskontrakt.

- Lærebøgerne forklarer bedre, hvorfor skalering hjælper optimeringen, men er ikke den mest præcise kilde til API-adfærd.

I rapporten: [5] citeres direkte ved StandardScaler-formlen og eksporten af scalerens værdier til C++.

Reference [6] – scikit-learn MLPClassifier

Scikit-learns officielle user guide beskriver den modelklasse, der faktisk trænes.

- Den beskriver et feedforward MLP med supervised træning.
- Den forklarer backpropagation og de understøttede aktiveringsfunktioner.
- Ved flerklassifikation anvendes softmax på outputtet.
- Dokumentationen er valgt til implementeringspåstande, fordi projektet bruger `MLPClassifier` direkte.
- En generel neural-netværksbog kan forklare matematikken dybere, men ikke nødvendigvis scikit-learns konkrete standarder og attributter.

I rapporten: [6] citeres ved beskrivelsen af MLP, backpropagation og softmax.

Reference [7] – Hands-On Machine Learning

Aurélien Géron's *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, tredje udgave, er bred baggrundslitteratur til den samlede ML-proces.

Relevante kapitler i den lokale PDF:

- Kapitel 1, PDF-side 31: machine-learning-landskabet.
- Kapitel 2, PDF-side 67: et end-to-end machine-learning-projekt.
- Kapitel 3, PDF-side 131: klassifikation, confusion matrix, precision og recall.
- Kapitel 4, PDF-side 159: træning, loss og optimering.
- Kapitel 7, PDF-side 239: ensembles og random forests.
- Kapitel 9, PDF-side 287: unsupervised learning og clustering.
- Kapitel 10, PDF-side 327: neuroner, MLP, backpropagation, ReLU og softmax.
- Kapitel 15, PDF-side 565: sekvensbehandling med blandt andet CNN'er.
- Kapitel 17, PDF-side 663: autoencodere.
- Appendix A, PDF-side 807: projektcheckliste fra problem til deployment.
- Bogen er valgt, fordi den forbinder teori med praktiske scikit-learn-workflows.
- Den er mere pædagogisk og sammenhængende end en samling API-sider.
- Den bruger også Keras og TensorFlow, som ikke er projektets deploymentteknologi.
- Derfor bruges den bedst til generelle principper, ikke som bevis for den konkrete firmware.

Vigtig præcision: [7] står i rapportens litteraturliste, men er ikke citeret ved en bestemt sætning i brødteksten. Den bør omtales som baggrundslitteratur.

Reference [8] – scikit-learn-artiklen

Pedregosa et al. fra *Journal of Machine Learning Research* introducerer scikit-learn som et open-source machine-learning-bibliotek til Python.

- Artiklen er en fagfællebedømt kilde til bibliotekets akademiske oprindelse og design.
- Den er ikke den bedste kilde til nutidige detaljer om en bestemt API-metode.
- Officiel API-dokumentation er mere præcis for `StandardScaler` og `MLPClassifier`.
- Artiklen kan begrunde valg af scikit-learn som etableret værktøj, men ikke projektets målte accuracy.

Vigtig præcision: [8] står i litteraturlisten, men er ikke citeret ved en bestemt sætning i rapportens brødtekst.

Reference [9] – Machine Learning with PyTorch and Scikit-Learn

Raschka, Liu og Mirjalilis *Machine Learning with PyTorch and Scikit-Learn* er særlig relevant til preprocessing, evaluering og neural-netværksfundamentet.

Relevante kapitler i den lokale PDF:

- Kapitel 1, PDF-side 30: supervised, unsupervised og reinforcement learning samt ML-workflow.
- Kapitel 3, PDF-side 82: klassifikationsalgoritmer i scikit-learn.
- Kapitel 4, PDF-side 134: train/test-split, feature scaling og feature selection.
- Kapitel 6, PDF-side 200: k-fold-cross-validation, hyperparametre og metrics.
- Kapitel 7, PDF-side 234: ensemble learning.
- Kapitel 10, PDF-side 334: K-means og clustering.
- Kapitel 11, PDF-side 364: MLP implementeret fra bunden, forward-pass og backpropagation.
- Kapitel 12, PDF-side 398: neurale netværk og aktiveringsfunktioner.
- Kapitel 14, PDF-side 480: convolutional neural networks.
- Kapitel 15, PDF-side 528: sekventielle data.
- Bogen er valgt, fordi dens scikit-learn-kapitler ligger tæt på projektets faktiske workflow.
- Den forklarer matematik og metode dybere end API-dokumentationen.
- PyTorch-delene er baggrundsviden og blev ikke brugt til implementeringen.

Vigtig præcision: [9] står i rapportens litteraturliste, men er ikke citeret ved en bestemt sætning i brødteksten. Den bør omtales som baggrundslitteratur.

Reference [10] og [11] – AI-værktøjer

BlackBox AI's og OpenAI Codex' officielle sider bruges kun til transparent at identificere de anvendte hjælpeværktøjer.

- De er ikke kilder til sensorens specifikationer.
- De er ikke kilder til projektets målinger eller modelresultater.
- De dokumenterer ikke, at AI-genereret kode er korrekt.
- Korrekthed blev i stedet kontrolleret gennem kodegennemgang, data, træning, build og fysisk test.

Valg af kildetype

Den korte prioritering er:

1. **Kursuskrav:** Kursets egen projektbrief.
2. **Hardwaretal:** Producentens datablad.
3. **Konkret softwareadfærd:** Officiel API- eller user-guide.
4. **Generel teori:** Anerkendte lærebøger og fagfællebedømte artikler.
5. **Egne resultater:** Reproducerbar kode, data, logs og fysisk måling.

Den samme kilde er ikke bedst til alt. Et datablad er stærkt til sensorens elektriske specifikationer, men forklarer ikke cross-validation. En lærebog forklarer backpropagation, men dokumenterer ikke Photon 2's RAM. Officiel scikit-learn-dokumentation forklarer API'en, men beviser ikke, at den deployede C++-kode er numerisk identisk.

Spørgsmålszoner i præsentationen

1. Efter **problem, krav og hardware**.
2. Efter **dataindsamling og preprocessing**.
3. Efter **model, matematik og deployment**.
4. Efter **resultater, begrænsninger og konklusion**.

Forslag til indledning:

"Jeg har struktureret præsentationen kronologisk. Jeg er autistisk, og pludselige kontekstskift kan gøre det svært for mig at fastholde strukturen, selv når jeg kender stoffet. Det hjælper mig, hvis spørgsmål så vidt muligt kommer ved mine fire tydelige stoppunkter. Jeg holder selv en kort pause ved hvert af dem."